

Ain Shams University
Faculty of Engineering
Computer Engineering and Software Systems



CFI - Care: Collaborative Flow of Intelligence for Care

Graduation Project Thesis

Author: Mohamed Salah Fathy 21P0117
Youssef Tamer Hossam 21P0138
Salma Hisham Hassan Wagdy 21P0124
Salma Mohamed Abdelmonem 21P0148
Belal Anas Awad 21P0072
Supervisors: Dr Haitham Azmi

Submission Date: 04 April, 2025

Mohamed Salah Fathy 21P0117
Youssef Tamer Hossam 21P0138
Salma Hisham Hassan Wagdy 21P0124
Salma Mohamed Abdelmonem 21P0148
Belal Anas Awad 21P0072
Date: 04 April, 2025

Abstract

Healthcare today is characterized by fragmented experiences where patient medical information is often dispersed across various institutions and providers. This project proposes a comprehensive, patient-centric healthcare application designed to consolidate all medical reports into a single, secure, and unified platform. The system employs Role-Based Access Control (RBAC) to ensure privacy and controlled information sharing. To enhance clinical decision-making, the platform features a medical history visualization graph for tracking health trends and integrates a Retrieval-Augmented Generation (RAG)-enabled AI model to generate diagnostic reports and insights. By transforming isolated data into an interconnected ecosystem, this project aims to establish meaningful relationships between patients, caregivers, and doctors for personalized healthcare delivery.

Contents

1	Introduction	1
1.1	Digital Health Market Overview	3
1.2	HL7 - Health Level Seven	3
1.2.1	Evolution of Standards	3
1.3	Fast Healthcare Interoperability Resources (FHIR)	4
1.3.1	Core Architecture	4
1.3.2	Key Technical Advantages	4
1.3.3	Comparison of Standards	5
2	Literature Review	7
2.1	Authentication and Authorization Standards	7
2.1.1	OAuth 2.0 and OIDC	7
2.1.2	JSON Web Tokens (JWT)	7
2.1.3	PKCE	7
2.2	Healthcare Security and Standards	7
2.2.1	FHIR and SMART on FHIR	7
2.3	Token-Based Authentication Patterns	8
2.3.1	Session Management	8
2.4	API Security	8
2.4.1	Rate Limiting	8
2.4.2	Input Validation and XSS	8
2.4.3	CSRF	8
2.5	Mobile Application Security	8
2.5.1	Secure Token Storage	8
2.6	Audit Logging and Compliance	8
2.6.1	Security Event Logging	8
2.6.2	Threat Modeling	9
2.7	Infrastructure and Deployment	9
2.7.1	API Gateway	9
3	Proposed Design Architecture	10
3.1	System Architecture	10
3.1.1	High-Level Architecture	10
3.1.2	Node.js Application Server	11

3.1.3	Data Persistence and Schema	12
3.1.4	Request/Response Flow and Data Transformation Pipeline	14
3.1.5	Error Handling and Fault Tolerance	19
3.1.6	Security and Infrastructure Layer	21
3.1.7	HAPI FHIR Server Implementation	21
3.2	API Design and Endpoints	22
3.2.1	RESTful Structure and Formats	22
3.2.2	FHIR-Compliant Endpoints	22
3.2.3	Custom History Graph Endpoints	22
3.3	Patient History Graph Implementation	23
3.3.1	Motivation: Beyond Linear Medical Records	23
3.3.2	Graph Data Model	25
3.3.3	Graph Construction Algorithm	25
3.3.4	Temporal Branching and Concurrent Episodes	27
3.3.5	Relationship Classification	28
3.3.6	Handling Edge Cases	29
3.3.7	Query Patterns and Optimization	29
3.4	Performance Optimization	30
3.4.1	Tier 1: Standard Resource Cache	31
3.4.2	Tier 2: Specialized Graph Cache	33
3.5	Frontend	36
3.6	Mobile	36
3.7	Problem Statement	36
3.7.1	Data Fragmentation and the “Silo” Effect	36
3.7.2	Reliance on Analog Systems and Data Quality	37
3.7.3	Absence of Standardized Consent and Data Sovereignty	37
3.7.4	The Need for Continuous Monitoring	37
3.8	Mobile Application Goal and Methods	37
3.8.1	Digital Document Management	37
3.8.2	Doctor-Patient Data Sharing	38
3.8.3	Elimination of Physical Records	38
3.8.4	Caregiver and Family Involvement	38
3.8.5	Appointment Management	38
3.8.6	Wearable Device Integration	38
3.9	Related Work	38
3.9.1	Apple Health	39
3.10	Market Analysis: The Egyptian Landscape	40
3.10.1	Vezeeta	40
3.10.2	Lumi by Nextcare:	40
3.10.3	Our Competitive Advantage: The ”Medical History Tree”	40
3.11	Flutter and Dart	41
3.11.1	Clean Architecture in the Mobile Application	41
3.11.2	Layered Structure	42
3.11.3	Centralized Runtime Permission Handling	42

3.12	Requirements Engineering	46
3.12.1	Target Group and User Group	47
3.12.2	Stakeholder	47
3.13	System Context Diagram	50
3.14	System Requirements Specification	50
3.14.1	Functional Requirements	51
3.14.2	Non-Functional Requirements	51
3.14.3	Optional Requirements (Future Scope)	52
3.15	Use Cases	52
3.15.1	SignIn and SignUp Pages	52
3.15.2	Document Management	52
3.15.3	Medical History Navigation	53
3.15.4	Appointment Booking	56
3.16	My Profile Update	58
3.17	Quality Assurance and Testing	59
3.17.1	Unit Testing	59
3.17.2	Widget Testing	60
3.17.3	Integration Testing	60
3.17.4	Testing Architecture and Tools	60
3.17.5	Unit Testing Implementation	61
3.18	Future Work	63
3.18.1	Push Notifications	63

Chapter 1

Introduction

Today, patients often face fragmented healthcare experiences, with their medical information dispersed across multiple institutions and providers. This project proposes a patient-centric healthcare application designed to consolidate all medical reports within a single, secure, and unified platform.

The system empowers patients to manage their health through continuous data collection, secure communication, and controlled information sharing using Role-Based Access Control (RBAC) to ensure privacy, consent, and data integrity. Additionally, by integrating features such as medical history visualization and Retrieval-Augmented Generation (RAG)-enabled AI, the proposed system transforms previously scattered and isolated medical data into a unified, interconnected ecosystem. This integration establishes meaningful relationships between patients, healthcare providers, and medical records, promoting continuous, coordinated, and personalized healthcare delivery.

Purpose

The primary purpose of this project is to develop a unified healthcare platform that securely consolidates patients' medical records from multiple institutions. By leveraging technologies such as RAG-enabled AI and graphical data visualization, the project aims to enhance clinical insight and support medical decision-making. Furthermore, the platform seeks to empower patients to monitor their own health data while maintaining full control over information sharing through robust RBAC mechanisms. Ultimately, this initiative strives to ensure privacy, data integrity, and interoperability across the healthcare ecosystem to enhance patient trust and system reliability.

Objective

The key objectives of this project are as follows:

- To develop a unified healthcare platform that securely consolidates patients' medical records from multiple institutions.
- To empower patients to manage and monitor their own health data while maintaining full control over information sharing through RBAC mechanisms.
- To enable secure and efficient communication between patients, doctors, and caregivers.
- To provide data visualization tools, including a medical history graph, to help healthcare providers interpret patient data trends and clinical progress.
- To integrate RAG-enabled AI capabilities to generate automated reports and actionable insights that support doctors in diagnosis and medical decision-making.
- To ensure privacy, data integrity, and interoperability across the healthcare ecosystem to enhance patient trust and system reliability.
- To implement a defense-in-depth security architecture leveraging OAuth 2.0, JWT, and FHIR Consent resources for multi-layered authentication, authorization, and access control across browser and mobile clients.
- To establish comprehensive audit logging, session management, and revocation capabilities for forensic analysis and immediate access termination.

1.1 Digital Health Market Overview

The global push towards digital transformation in healthcare is driven by the need for efficiency, accessibility, and improved patient outcomes. According to Grand View Research, the global digital health market size was valued at USD 288.55 billion in 2024 and is projected to reach USD 946.04 billion by 2030, expanding at a Compound Annual Growth Rate (CAGR) of 22.2% from 2025 to 2030 [?].

This rapid growth is fueled by factors such as the increasing penetration of smartphones, advancements in internet connectivity, and government initiatives promoting digital health infrastructure [?]. However, realizing this potential requires overcoming the challenge of data silos. Currently, healthcare data is often locked within proprietary systems, making *interoperability*—the ability of different information systems to access, exchange, integrate, and cooperatively use data in a coordinated manner—a critical requirement for the modern healthcare ecosystem [?].

1.2 HL7 - Health Level Seven

Health Level Seven (HL7) International is a standards development organization that has defined the framework for the exchange, integration, sharing, and retrieval of electronic health information for decades [?]. Understanding the evolution of these standards is crucial to appreciating the architectural shifts in modern healthcare applications.

1.2.1 Evolution of Standards

- **HL7 v2 (The Workhorse):** Established in 1989, HL7 v2 remains the most widely implemented standard for hospital workflows (e.g., ADT - Admission, Discharge, Transfer messages). It utilizes a non-XML, pipe-delimited syntax (e.g., MSH|^~\&|...). While highly efficient for legacy backend systems, its lack of semantic strictness leads to significant implementation variances, posing challenges for modern web and mobile application development [?].
- **HL7 v3 (The Theoretical Model):** Introduced in the 1990s to address the vagueness of v2, HL7 v3 was built on a rigorous Reference Information Model (RIM). It enforced strict semantic correctness using complex XML structures. However, this theoretical purity resulted in a steep learning curve and bloated message sizes, leading to poor adoption rates and a lack of backward compatibility with v2 [?].
- **HL7 CDA (Clinical Document Architecture):** A component of v3, CDA focuses on the exchange of clinical documents (such as discharge summaries). While effective for static snapshots of care, it is document-centric rather than data-centric, making it difficult to extract granular data (like a single heart rate reading) required for real-time interactive applications [?].

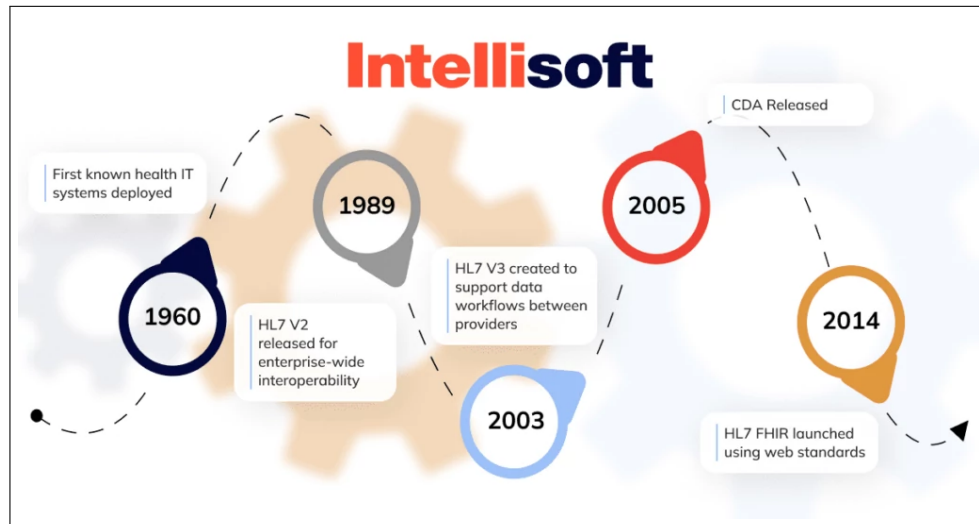


Figure 1.1: Evolution of HL7 Standards from Messaging to RESTful Resources.

1.3 Fast Healthcare Interoperability Resources (FHIR)

In response to the complexity of HL7 v3 and the limitations of v2, HL7 introduced **FHIR** (Fast Healthcare Interoperability Resources) in 2012. FHIR represents a paradigm shift from complex, custom healthcare protocols to modern web standards.

1.3.1 Core Architecture

Unlike its predecessors, FHIR is built on the RESTful API approach widely used in modern IT industries (e.g., by Google, Facebook, Twitter). It treats data elements as independent "Resources"—modular components like **Patient**, **Observation**, or **Medication**—that can be accessed and manipulated via standard HTTP methods (GET, POST, PUT, DELETE) [?].

1.3.2 Key Technical Advantages

- **Granularity via Resources:** FHIR allows systems to access specific data elements using unique URLs (e.g., `GET /Patient/123`), rather than exchanging entire heavy documents. This makes it ideal for mobile devices with limited bandwidth [?].
- **Modern Formats (JSON & XML):** While previous standards relied on complex XML or pipe-delimited text, FHIR natively supports JSON (JavaScript Object Notation), which is lightweight and directly parsable by web browsers and mobile apps [?].

- **The 80/20 Rule:** FHIR focuses on defining the 20% of requirements that satisfy 80% of common healthcare use cases. For the remaining complex or local requirements, it uses a built-in extension mechanism, ensuring the core standard remains simple and implementable [?].

1.3.3 Comparison of Standards

Table 1.1 highlights the fundamental differences between the major HL7 standards, illustrating why FHIR was chosen as the interoperability backbone for this project.

Table 1.1: Comparison of HL7 v2, v3, CDA, and FHIR Standards

Feature	HL7 v2	HL7 v3 / CDA	FHIR
Primary Paradigm	Event-Driven Messaging	Reference Model (RIM) / Documents	RESTful API & Resources
Data Format	Pipe-delimited text	Complex XML	JSON, XML, RDF
Transport	LLP / MLLP (TCP/IP)	SOAP / Web Services	HTTP / REST
Complexity	Moderate (Vague semantics)	High (Steep learning curve)	Low (Web-standard based)
Use Case	Hospital Backend (ADT, Orders)	Clinical Documents (Summaries)	Mobile, Web, Cloud, & Backend

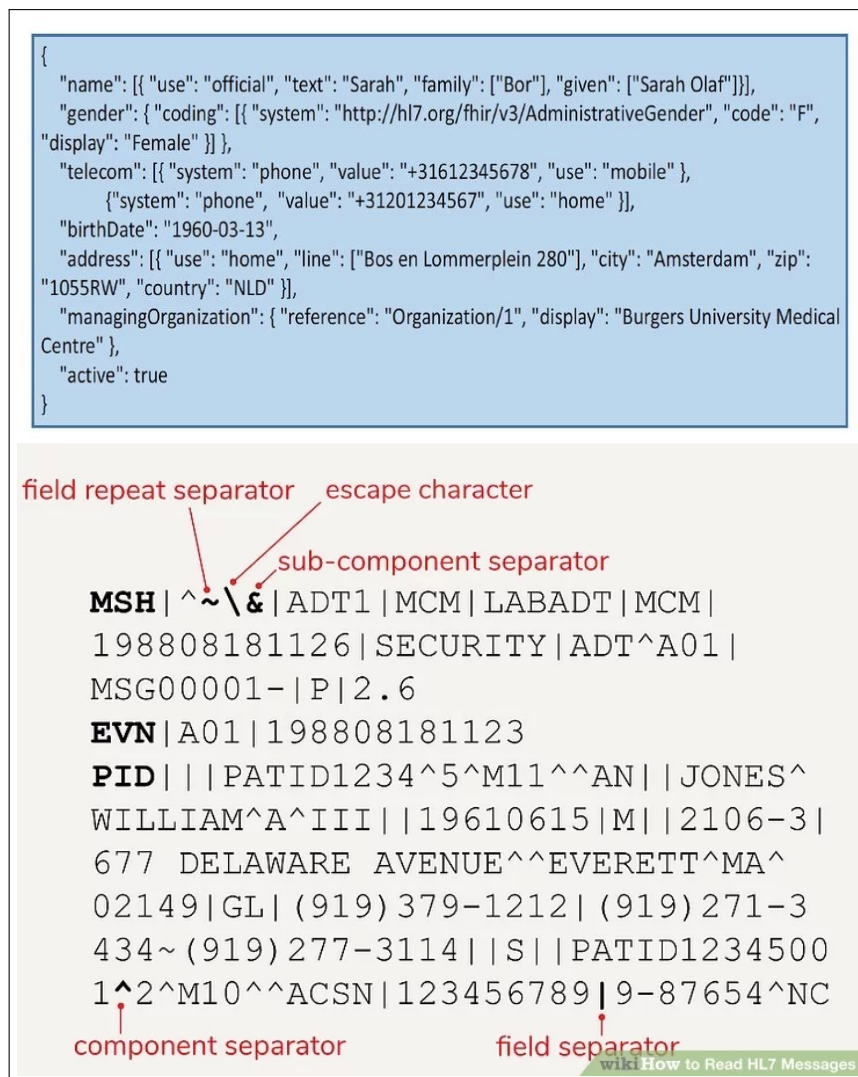


Figure 1.2: Visual Comparison of Data structure: HL7 v2 Message vs. FHIR JSON Resource.

Chapter 2

Literature Review

2.1 Authentication and Authorization Standards

2.1.1 OAuth 2.0 and OIDC

OAuth 2.0 [?] defines authorization grant flows: Authorization Code (web), Resource Owner Password (custom UI), Client Credentials (service-to-service). OpenID Connect (OIDC) [?] adds an identity layer with ID tokens and SSO. CFI-Care uses OAuth 2.0 Resource Owner Password flow with OIDC for token-based API access.

2.1.2 JSON Web Tokens (JWT)

JWTs [?] are signed tokens encoding claims (iss, aud, exp, roles, scopes). Stateless API authentication validates signatures without database queries. Token revocation requires server-side tracking or short expiration windows.

2.1.3 PKCE

PKCE [?] prevents authorization code interception attacks via code verifier/challenge. Recommended for all OAuth 2.0 flows [?].

2.2 Healthcare Security and Standards

2.2.1 FHIR and SMART on FHIR

FHIR [?] defines RESTful APIs for healthcare data exchange. SMART on FHIR [?] applies OAuth 2.0 to FHIR with scoped access (e.g., `patient/Observation.read`). The FHIR consent resource [?] models patient-controlled access with time-bound and actor-specific rules.

2.3 Token-Based Authentication Patterns

2.3.1 Session Management

HTTP-only cookies prevent JavaScript access [?]; SameSite blocks CSRF. token revocation uses redis-backed session tracking or OAuth 2.0 server revocation.

2.4 API Security

2.4.1 Rate Limiting

Rate limiting mitigates brute-force and enumeration attacks via Redis-backed counters [?].

2.4.2 Input Validation and XSS

Schema validation (Joi) enforces format/complexity rules. OWASP [?] recommends output encoding and CSP headers.

2.4.3 CSRF

CSRF [?] is mitigated with SameSite cookies and CSRF tokens.

2.5 Mobile Application Security

2.5.1 Secure Token Storage

Android Keystore [?] provides OS-level encryption and access control, unlike browser localStorage.

2.6 Audit Logging and Compliance

2.6.1 Security Event Logging

OWASP [?] specifies what to log and exclude audit logs enable forensics and compliance.

2.6.2 Threat Modeling

RFC 6819 [?] outlines OAuth 2.0 threats: token theft, interception, phishing, CSRF, replay.

2.7 Infrastructure and Deployment

2.7.1 API Gateway

nginx[?] terminates TLS, routes requests, enforces rate limits, and sets security headers.

Chapter 3

Proposed Design Architecture

3.1 System Architecture

The CFI-Care system is built upon a robust three-tier architecture designed to facilitate secure and interoperable healthcare data management. The system integrates a modern frontend, a logic-heavy middleware layer, and a standards-compliant backend to ensure data consistency and security. The architecture adheres to the FHIR (Fast Healthcare Interoperability Resources) standard for all medical data exchange and utilizes OAuth 2.0 via Keycloak for identity management.

3.1.1 High-Level Architecture

The application operates through a strict separation of concerns. The Presentation Layer is an Angular-based Single Page Application (SPA) that serves as the user interface. It does not communicate directly with the data stores. Instead, all client requests are intercepted by an Nginx Reverse Proxy, which handles SSL/TLS termination and routes traffic to the appropriate internal services.

The core of the backend processing is handled by the Node.js Application Server. This server acts as the primary API gateway, implementing business logic, enforcing security policies, and managing user sessions. It communicates with the HAPI FHIR Server, a specialized Java-based component responsible for persisting clinical data in compliance with the FHIR R5 standard. Data storage is consolidated in PostgreSQL databases, while high-speed data access and session management are handled by Redis instances.

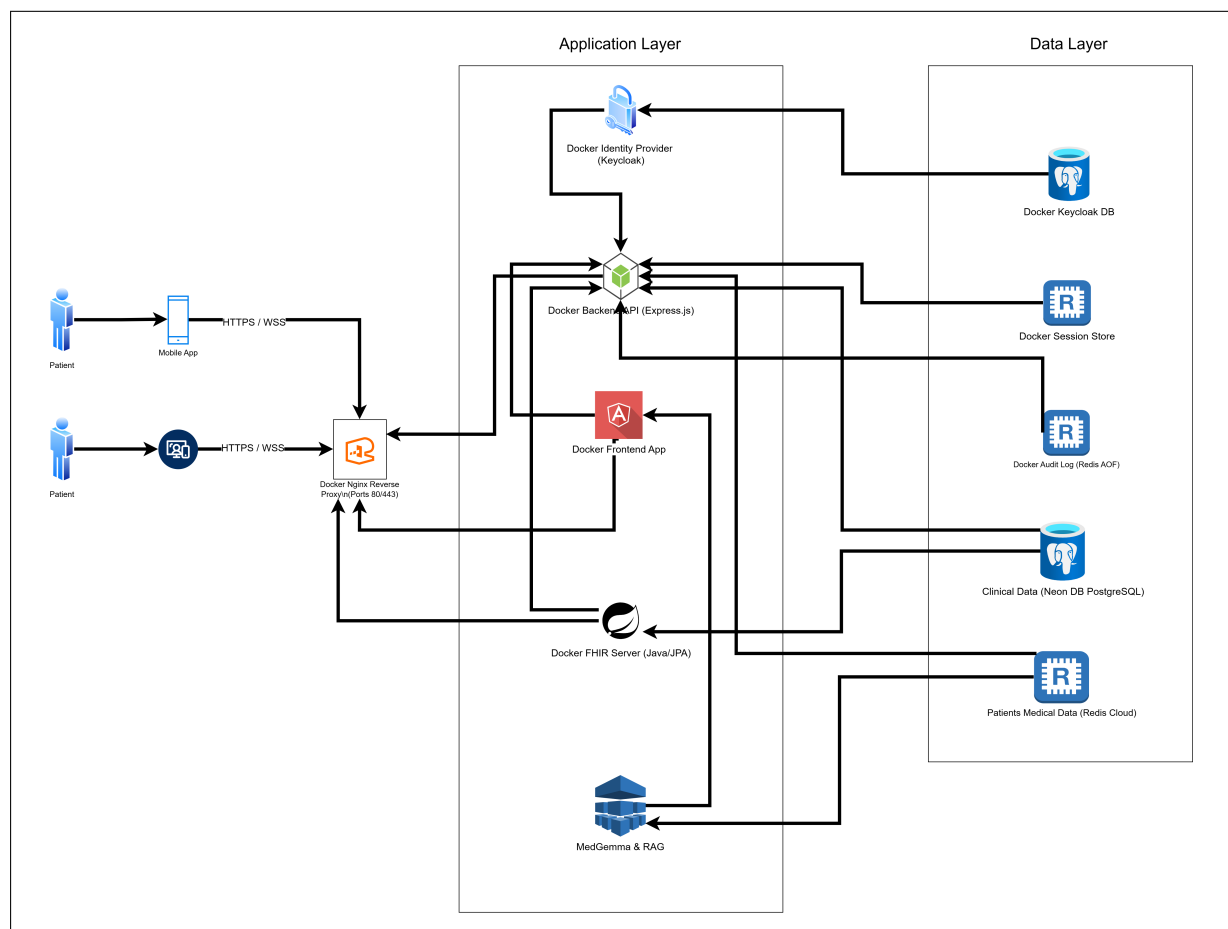


Figure 3.1: High-Level Container Architecture showing the interaction between the Nginx proxy, application services, and data persistence layers.

3.1.2 Node.js Application Server

The Node.js server, built with the Express.js framework, serves as the middleware layer that bridges the gap between the user interface and the standardized medical records. It is designed with a modular layered architecture to ensure scalability and maintainability.

Middleware and Security

Upon receiving a request, the server passes it through a comprehensive middleware stack. Security headers are applied using Helmet to prevent common web vulnerabilities. Cross-Origin Resource Sharing (CORS) is configured to restrict access to authorized domains. Crucially, the server implements rate limiting to protect API endpoints from abuse and utilizes a Redis-backed session store to manage user authentication states efficiently.

Routing and Services

The application is divided into specific route modules corresponding to healthcare domains. Dedicated endpoints manage Patients, Practitioners, Encounters, and Episodes of Care. A specialized "History Graph" module handles the retrieval and processing of graph visualization data.

Beneath the routing layer lies the service layer, which contains the core business logic. These services act as proxies to the FHIR server, transforming raw FHIR JSON into formats optimized for the frontend, such as the D3.js visualizations. They also handle document processing tasks, such as converting base64 strings to PDF files for binary storage.

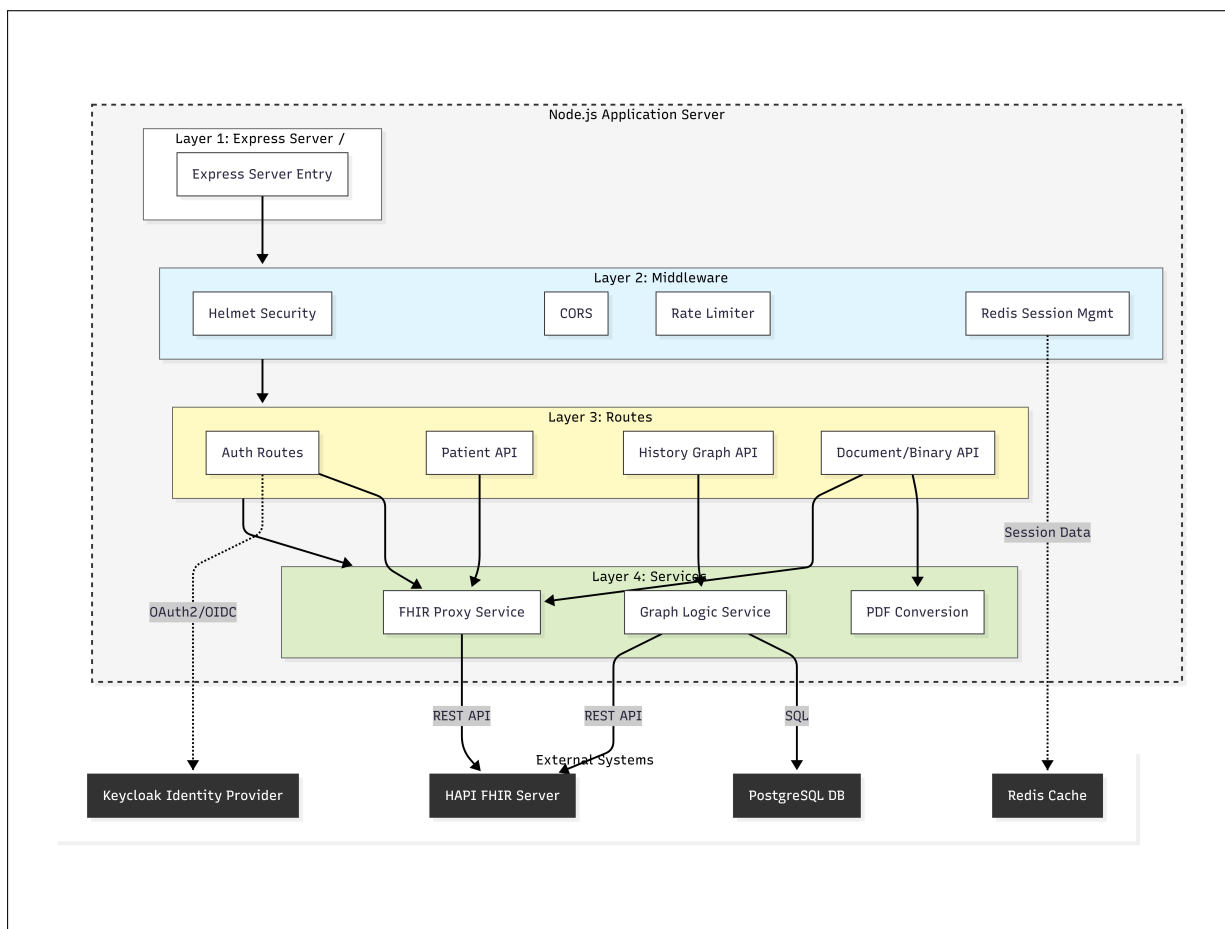


Figure 3.2: Internal layered architecture of the Node.js Application Server, detailing the middleware stack and service distribution.

3.1.3 Data Persistence and Schema

The system utilizes a polyglot persistence strategy, employing different database technologies for specific use cases to maximize performance.

Relational Data

The primary storage engine is PostgreSQL. It is divided into two logical segments. The first segment supports the HAPI FHIR server, storing all standard clinical resources (like Patients and Observations) in a highly structured format managed by Hibernate. The second segment supports the Keycloak identity provider, storing user credentials and realm configurations.

Custom Graph Schema

To support the unique "Medical History Graph" feature, a custom schema was designed alongside the standard FHIR tables. This schema consists of **encounter_nodes** and **node_relations**. These tables allow the system to map standard FHIR Encounters into a Directed Acyclic Graph (DAG), enabling the visualization of a patient's journey through time with causal links between medical events.

Caching and Auditing

Redis is utilized for ephemeral data storage. One instance is dedicated to storing user sessions, ensuring fast authentication checks. A second Redis instance operates in Append-Only File (AOF) mode to store audit logs, providing a persistent and high-speed trail of system activities.

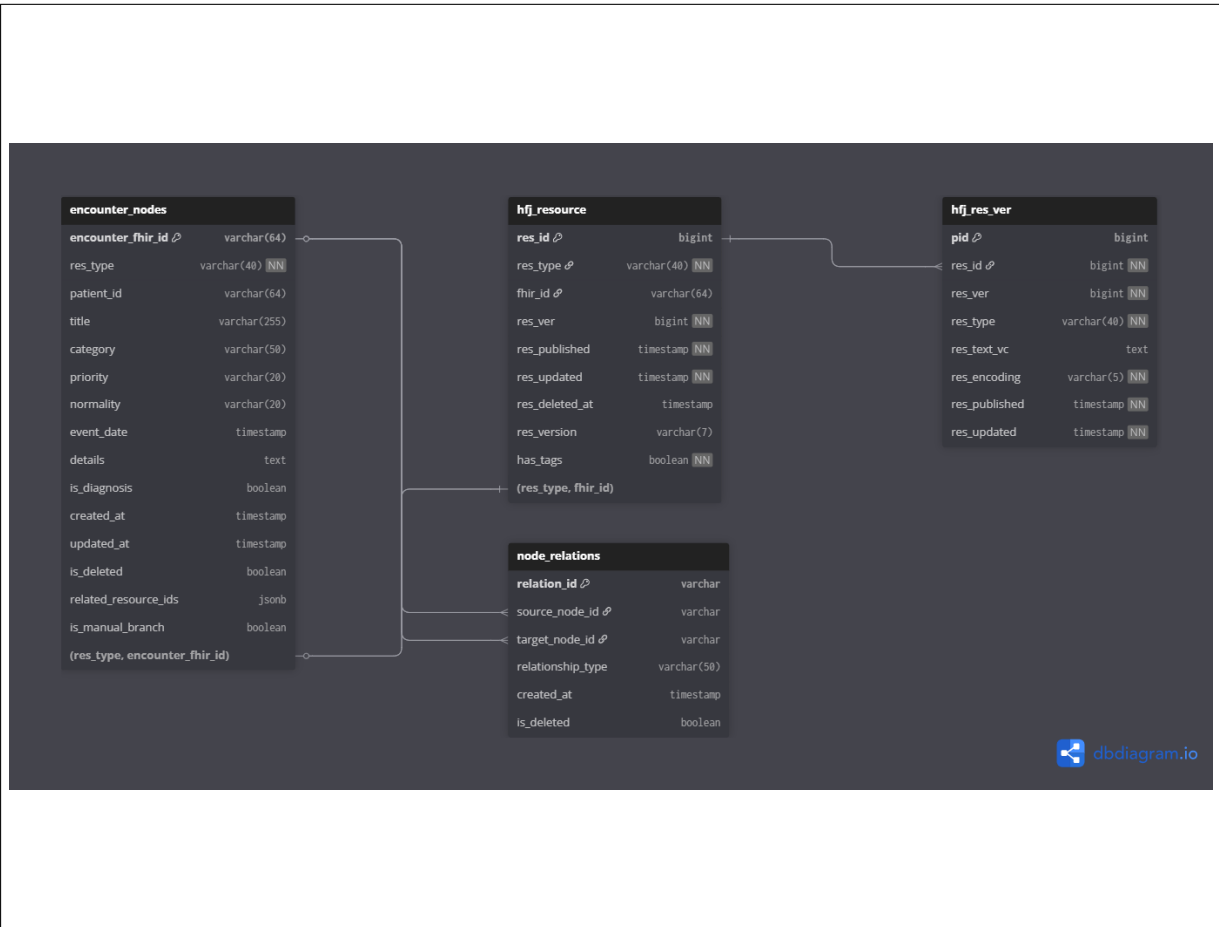


Figure 3.3: Entity-Relationship Diagram (ERD) illustrating the integration of custom graph tables with the standard FHIR resource table.

3.1.4 Request/Response Flow and Data Transformation Pipeline

Understanding how requests traverse the CFI-Care system is essential for comprehending the architecture’s design decisions. A typical request from the frontend follows a well-defined path through multiple layers, each contributing specific functionality: authentication verification, input validation, business logic execution, data transformation, and response formatting.

Request Ingress and Authentication

A request originates from the Angular frontend as an HTTP request, typically including a Bearer token in the Authorization header. The request first reaches the Nginx reverse proxy, which acts as the external-facing gateway. Nginx terminates the TLS/SSL connection, decrypts the HTTPS payload, and forwards the unencrypted request to the Node.js server over the private Docker network. Before the request reaches application

logic, it passes through the Helmet middleware, which applies security headers to prevent common web vulnerabilities such as XSS (Cross-Site Scripting), CSRF (Cross-Site Request Forgery), and clickjacking. The CORS middleware then validates that the request originates from an authorized frontend domain; requests from unauthorized origins are rejected immediately.

The next critical middleware validates the JWT token embedded in the Authorization header. The Node.js server maintains a cached set of public keys (JWKS) from Keycloak, updated periodically to handle key rotations. The token is cryptographically verified against these keys. If verification succeeds, the token claims (user ID, roles, permissions) are extracted and attached to the request context. If verification fails—due to an invalid signature, expired token, or missing token—the middleware returns a 401 Unauthorized response, and the request is rejected before reaching downstream services.

Concrete Example: Patient Data Retrieval

Consider a clinician’s request to retrieve a patient’s full medical history. The frontend issues a GET request to `/api/patients/:patientId`. After passing authentication and security middleware, the request reaches the routing layer, which dispatches it to the `PatientService` controller. The controller immediately implements input validation: the `patientId` parameter is checked to ensure it conforms to expected format (numeric ID or UUID, depending on the system design), and the authenticated user’s roles are verified to confirm they have permission to access that patient’s data (enforcing HIPAA privacy principles).

Before querying the database, the controller checks the Tier 1 cache (managed by `cacheHelper`). The middleware generates a deterministic cache key from the request URL: `cache:patients:patientId:{id}`. If a valid cache entry exists, the cached patient data is immediately returned with a 200 OK response, bypassing all downstream processing. This cache hit path typically completes in 20-50 milliseconds.

If a cache miss occurs, the request proceeds to the `PatientService` layer. The service constructs a FHIR query and forwards it to the HAPI FHIR server via an HTTP call: `GET /fhir/Patient/{patientId}`. The FHIR server retrieves the patient resource from PostgreSQL, performs any necessary transformations or computations required by the FHIR R5 standard, and returns the response in FHIR JSON format.

The `PatientService` receives the FHIR response and performs transformation operations: extracting relevant fields, formatting dates to match frontend expectations, computing derived values (e.g., age from birthdate), and sanitizing sensitive information if necessary. The transformed data is then stored in the Redis cache with a 1-hour TTL for future requests. Finally, the response is serialized to JSON and returned to the frontend with a 200 OK status code and appropriate cache headers to guide browser-side caching.

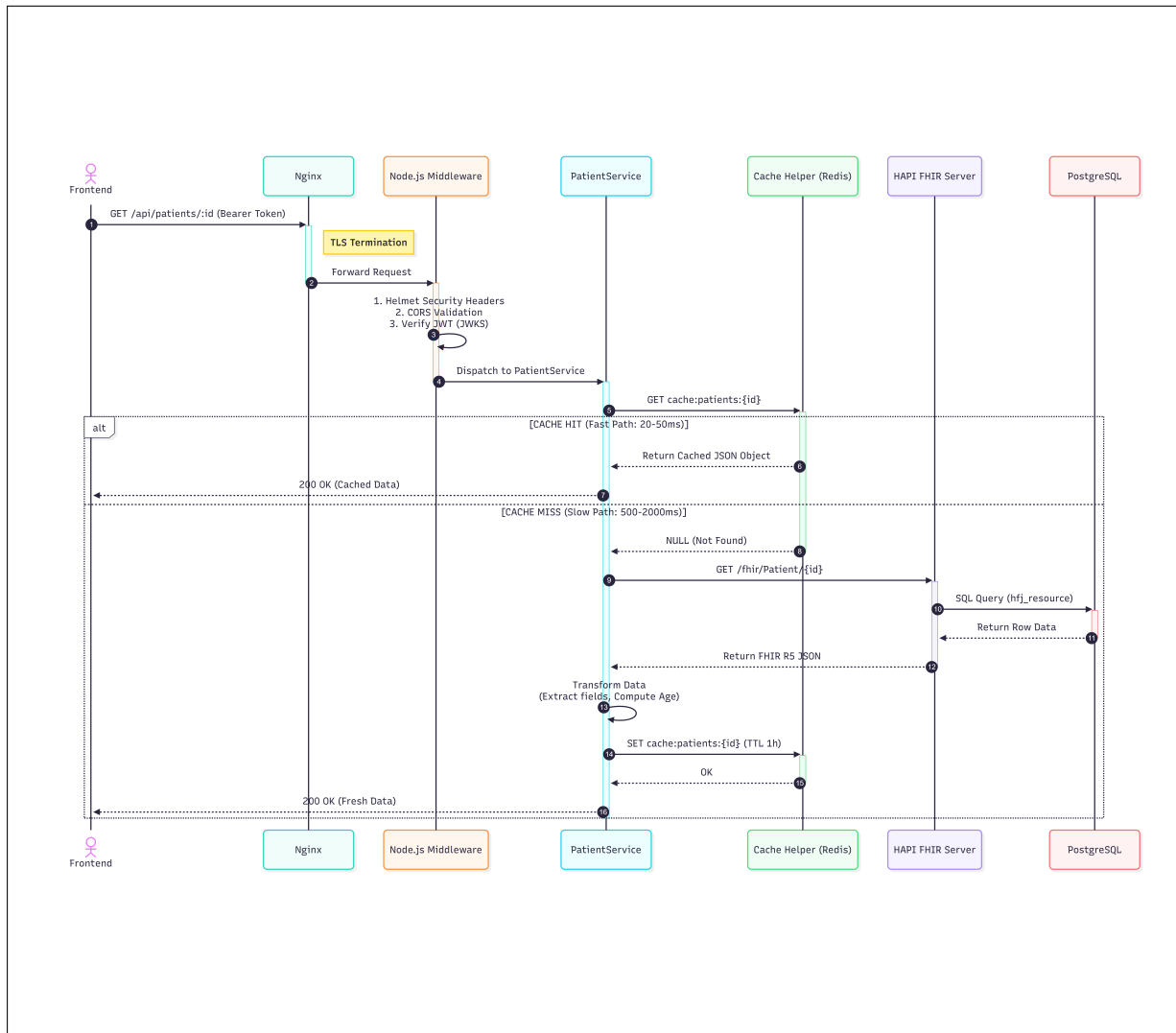


Figure 3.4: Request/Response flow for patient data retrieval, showing cache hit and miss scenarios with complete middleware pipeline.

Concrete Example: Encounter Creation and Graph Update

The encounter creation flow demonstrates more complex interactions across multiple system layers. When a clinician records a new consultation, the frontend issues a POST request to `/api/encounters` with a JSON payload containing encounter details (patient ID, encounter type, date, notes, etc.).

The request undergoes the same initial security and authentication pipeline as before. Upon reaching the `EncounterService` controller, the request enters the input validation phase. The encounter data is validated against a schema ensuring that all required fields are present and correctly formatted. The encounter date is verified to be a valid date and logically consistent (e.g., encounter date cannot be in the future). The patient ID is validated to ensure the patient exists in the system.

Next, the `EncounterService` constructs a FHIR Encounter resource conforming to the FHIR R5 standard and submits a POST request to the HAPI FHIR server: `POST /fhir/Encounter`. The FHIR server validates the resource, generates a unique FHIR resource ID, persists the resource to PostgreSQL, and returns the created resource with its new ID.

The encounter creation triggers a cascade of downstream operations. The `EncounterService` receives the FHIR response and extracts the newly created encounter's ID. It then invokes the `historyGraphService`, passing the new encounter data. The `historyGraphService` performs a temporal scan of existing nodes for the patient, identifies potential parent nodes based on clinical rules and temporal windows, and creates appropriate edges in the `node_relations` table. This graph construction may take several hundred milliseconds to several seconds depending on the patient's history size.

Concurrently, multiple cache entries are invalidated. The patient-specific Tier 2 graph cache is cleared immediately, ensuring that the next graph visualization request triggers a fresh reconstruction. The Tier 1 cache entry for the `/api/patients/{patientId}` endpoint is also invalidated, forcing a fresh FHIR server query for updated patient data.

Finally, the `EncounterService` returns the newly created encounter resource to the frontend with a 201 Created status code. The response includes the encounter's FHIR ID, allowing the frontend to reference this specific encounter in subsequent requests.

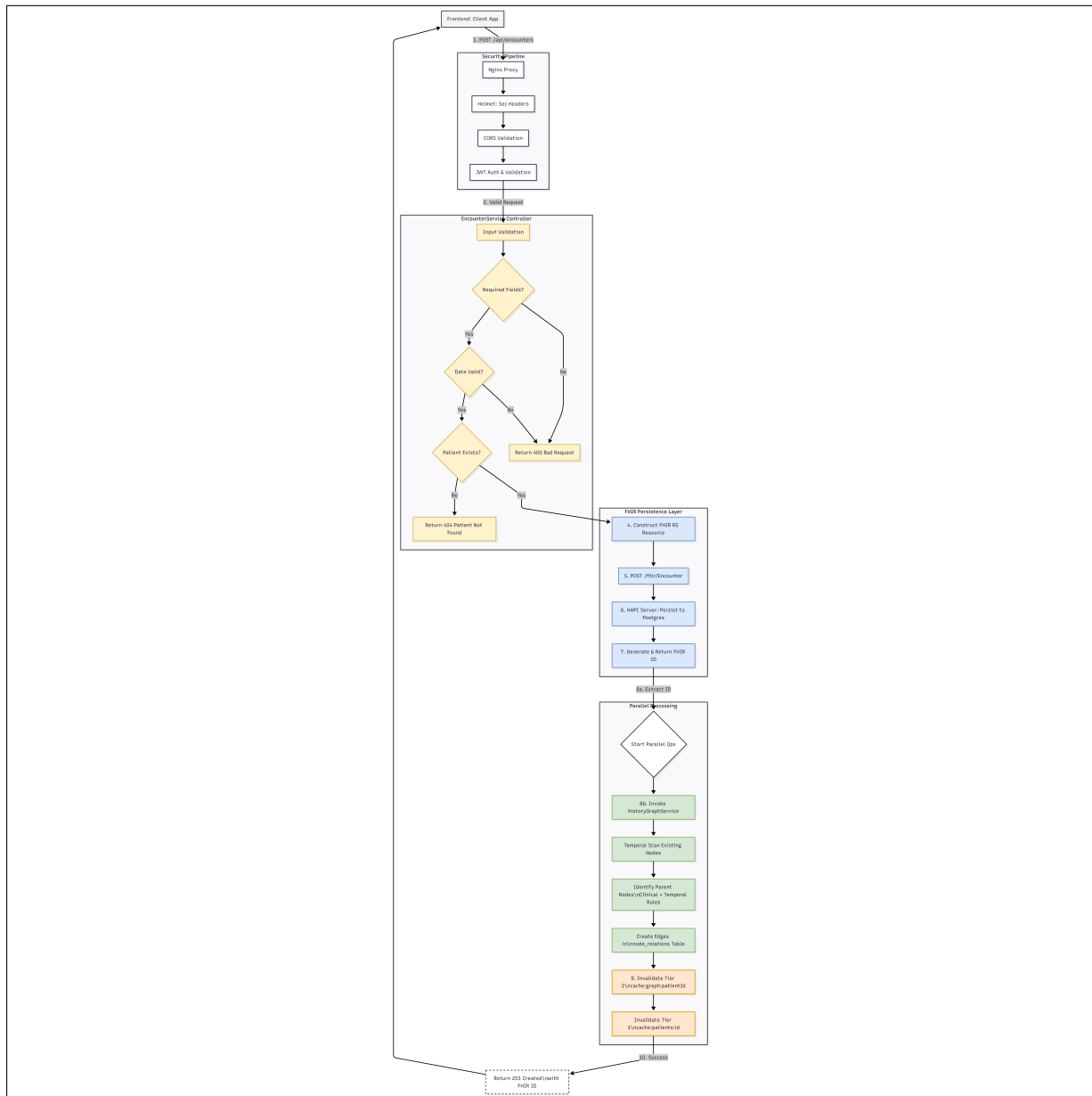


Figure 3.5: Complete encounter creation workflow showing FHIR resource persistence, graph construction, and cache invalidation cascade.

Validation Points and Error Conditions

Throughout this flow, validation occurs at multiple critical junctures. Input validation at the controller layer ensures that user-supplied data conforms to expected types and ranges. Business logic validation in the service layer verifies that operations are consistent with clinical rules (e.g., an encounter date cannot precede a patient’s birth date). FHIR resource validation at the HAPI FHIR server ensures that resources conform to the FHIR R5 specification, preventing invalid data from being persisted.

Error conditions at any stage result in appropriate HTTP status codes and error messages: 400 Bad Request for input validation failures, 403 Forbidden for authorization violations, 404 Not Found for missing resources, 409 Conflict for data consistency violations, and 500 Internal Server Error for unexpected server failures. Each error response includes a structured JSON object with a unique error code, human-readable message, and optionally, detailed information about the nature of the failure, enabling clinicians and developers to understand and address issues.

3.1.5 Error Handling and Fault Tolerance

The CFI-Care architecture recognizes that healthcare systems cannot tolerate downtime or data loss. Error handling and fault tolerance are designed into every layer to ensure reliable operation and graceful degradation under failure conditions.

Layered Error Handling Strategy

Errors propagate through the CFI-Care system in a controlled, layered manner. At the lowest layer, database operations (PostgreSQL queries, Redis operations) can fail due to connection timeouts, query errors, or resource unavailability. The Node.js middleware layer implements error handlers that catch these exceptions, log them for diagnostics, and transform them into appropriate HTTP error responses that safely communicate the failure to clients without exposing internal implementation details (such as database error messages or stack traces).

The service layer implements domain-specific error handling. When a FHIR server call fails, the service layer attempts to determine whether the failure is transient (temporary network issue) or persistent (FHIR server down, resource not found). For transient failures, the service implements exponential backoff retry logic: the first retry occurs after 100 milliseconds, the second after 200 milliseconds, and subsequent retries double the wait time, capped at a maximum of 30 seconds. This approach allows temporary outages to resolve while avoiding overwhelming a recovering server with immediate retry attempts. For persistent failures, errors are returned to the client immediately rather than consuming retry attempts.

The controller layer handles errors from services and formats them appropriately for the client. A service that cannot retrieve a patient record returns an error code; the controller translates this into a 404 Not Found HTTP response with descriptive messaging.

Fallback Mechanisms and Graceful Degradation

When the FHIR server becomes temporarily unavailable, the CFI-Care system implements fallback strategies to maintain partial functionality. The caching layer becomes

the primary data source: clinicians can still access patient records that were recently viewed, as these records remain in the Redis cache. While cached data may not reflect the absolute latest updates, it permits continued operation during brief FHIR server outages.

For the Patient History Graph, if the graph reconstruction fails due to FHIR server unavailability, the system returns a cached version of the graph if one exists, even if it is slightly stale. This allows clinicians to view previously computed graphs rather than receiving a complete failure. A notification is displayed to indicate that the graph may not reflect recent changes.

For new operations that require FHIR server access (creating encounters, adding diagnoses), if the FHIR server is unavailable, the system cannot proceed and returns a 503 Service Unavailable response, informing the clinician that the action cannot be completed at this moment. Rather than accepting incomplete or invalid data, the system fails safely, requiring the user to retry when the service recovers.

Resilience for Cache Operations

Redis provides the caching infrastructure for both session management and data caching. If Redis becomes unavailable, the system implements fallback strategies. For session caching, a secondary in-memory session store provides a fallback mechanism, allowing authentication to continue for users with active sessions, though new sessions cannot be created. For data caching, cache misses gracefully trigger direct queries to the FHIR server, bypassing the cache entirely but maintaining functionality.

Both Redis instances (session store and audit log store) are configured with persistence mechanisms. The session Redis instance uses RDB snapshots to periodically save state to disk, enabling recovery of active sessions if the instance restarts. The audit log Redis instance operates in AOF (Append-Only File) mode, ensuring every write is persisted to disk before acknowledging the write, guaranteeing that audit logs are never lost even if the Redis instance crashes.

Logging and Observability for Error Diagnosis

All errors are logged with structured logging that includes timestamp, error level (ERROR, WARN, INFO), service component generating the error, error code and message, request context (user ID, patient ID, operation being performed), and stack trace for unexpected exceptions. This structured logging enables rapid diagnosis of issues: operators can search logs by error code to identify all instances of a particular failure, by user ID to understand a user's experience during an outage, or by component to focus on specific services.

Critical errors are propagated to alerting systems, enabling operators to be notified of failures in real-time. For instance, if the FHIR server becomes unavailable, an alert is triggered within seconds. If error rates exceed thresholds, alerts notify operators that the system is degrading. This proactive monitoring enables rapid response to emerging issues before they impact clinicians' ability to deliver care.

3.1.6 Security and Infrastructure Layer

The infrastructure is secured through a centralized Identity and Access Management (IAM) system using Keycloak. Keycloak handles all OAuth 2.0 and OpenID Connect flows, issuing JSON Web Tokens (JWT) that the Node.js backend validates against a public key set (JWKS).

All components are containerized using Docker and orchestrated via Docker Compose. The environment is isolated within a private bridge network, with Nginx acting as the sole entry point. Nginx enforces SSL/TLS encryption using locally generated certificates, ensuring that all data in transit is encrypted. Containers are configured with read-only file system mounts and strict privilege limitations to minimize the security attack surface.

3.1.7 HAPI FHIR Server Implementation

The core component responsible for standardizing healthcare data within the CFI-Care system is the FHIR Server. This server is implemented using the HAPI FHIR JPA Server Starter framework. It is built upon the Java Spring Boot ecosystem and provides a fully compliant implementation of the FHIR Release 5 (R5) standard. This choice ensures that all clinical data, regardless of its source, adheres to international interoperability protocols and persists reliably within the database.

To tailor the server for the specific needs of this project, several important configuration changes were applied to the default setup. To maintain a lightweight and efficient runtime, advanced features that were not immediately required were explicitly disabled. This includes the Clinical Reasoning module and the CDS Hooks service. Similarly, bulk export and import operations were deactivated to prevent unnecessary resource consumption. Conversely, the OpenAPI documentation endpoints were enabled. This configuration allows developers to inspect the API structure and test endpoints directly through a web interface.

The persistence layer is powered by a PostgreSQL database connected via the Hibernate JPA engine. Specific configurations were applied to the Hibernate settings to optimize performance. The system uses the dedicated `HapiFhirPostgresDialect` to ensure efficient SQL generation. Additionally, the database schema management is configured to automatically update the table structures upon server startup, ensuring that the database remains synchronized with the code.

The server is responsible for managing a wide array of clinical resources necessary for the application. These include foundational entities such as Patient and Practitioner, as well as clinical events like Encounters, Procedures, and Observations. It also handles complex data types, including MedicationRequests for prescriptions and Binary resources for storing documents like PDF files.

For deployment, the server utilizes a secure multi-stage Docker build process. The first stage uses Maven to compile the application code and generate the necessary artifacts. The final runtime image is built upon a "distroless" Java 17 base image. This approach significantly reduces the final container size and improves security by removing unnecessary operating system tools and shells from the production environment.

3.2 API Design and Endpoints

The communication between the frontend client and the backend server is established through a structured RESTful API. This design ensures that the system remains stateless and scalable, utilizing standard HTTP methods to perform operations on resources. The Node.js application server acts as the centralized gateway, exposing a unified set of endpoints that handle both standard clinical data and custom graph logic.

3.2.1 RESTful Structure and Formats

All API interactions strictly adhere to the Javascript Object Notation (JSON) format for both request bodies and response payloads. The system utilizes standard HTTP verbs to define the nature of the operation. GET requests are used exclusively for retrieving data without side effects, while POST requests handle the creation of new resources. PUT methods are employed for updating existing records, and DELETE methods remove specific resources when permitted.

Responses from the server are standardized to ensure predictability for the client application. Successful operations return a 200 OK status code accompanied by the requested data object. In cases of resource creation, a 201 Created status is returned. Error handling is centralized, ensuring that any failure creates a consistent JSON error object containing a human-readable message and a specific error code, rather than exposing raw server stack traces.

3.2.2 FHIR-Compliant Endpoints

A significant portion of the API is dedicated to proxying standard FHIR resources. Endpoints such as `/api/patients` and `/api/encounters` act as a validation and caching layer sitting in front of the HAPI FHIR server. When the frontend requests patient details, the Node.js server intercepts this call, validates the parameters, and then forwards the request to the underlying Java server. The response is strictly compliant with the FHIR R5 standard, ensuring that the data structure remains interoperable with other external healthcare systems.

3.2.3 Custom History Graph Endpoints

Beyond standard FHIR resources, the API exposes specialized endpoints designed to support the Patient History Graph visualization. The primary endpoint, `/api/historyGraph`, does not return a simple list of records. Instead, it aggregates data from multiple FHIR resources and transforms them into a graph structure containing nodes and relational edges. This allows the frontend to render the complex temporal relationships between medical events without needing to perform heavy data processing in the browser.

Method	Endpoint	Description
GET	/api/patients	Retrieve list of patients with pagination support
POST	/api/patients	Register a new patient with FHIR validation
GET	/api/encounters/:id	Get specific encounter details including clinical observations
GET	/api/historyGraph/:id	Get patient history graph nodes and causal relationships
POST	/auth/login	Authenticate user and obtain JWT token via Keycloak

Table 3.1: Key REST API Endpoints provided by the Node.js Backend.

3.3 Patient History Graph Implementation

A core distinguishing feature of the CFI-Care backend is the Patient History Graph, which visualizes a patient’s medical journey as a Directed Acyclic Graph (DAG). Unlike standard FHIR lists, which present data linearly, this graph models causal and temporal relationships between medical events. This representation provides clinicians with a sophisticated tool for understanding the evolution of a patient’s condition, particularly in complex cases involving multiple concurrent diagnoses or extended treatment histories.

3.3.1 Motivation: Beyond Linear Medical Records

Traditional Electronic Health Records (EHRs) typically present patient data as chronologically ordered events: a consultation on January 1st, a lab test on January 3rd, a prescription on January 5th, and so forth. While this linear representation is straightforward to implement and understand, it obscures the causal relationships that are essential to clinical reasoning. Consider a patient with Type 2 Diabetes presenting with hypertension and elevated cholesterol levels. In a linear record, these appear as separate events without explicit connection. However, a clinician recognizes that these conditions are causally related—elevated cholesterol and hypertension may be comorbidities stemming from the underlying metabolic dysfunction of diabetes.

The Patient History Graph addresses this limitation by explicitly modeling causality and temporal dependencies. Each medical event is represented as a node in the graph, and directed edges represent causal or temporal relationships. This graphical representation allows clinicians to trace the progression of a specific condition across multiple encounters

and interventions, identify temporal clusters of related events that may indicate disease progression or treatment response, visualize the impact of therapeutic interventions on subsequent clinical outcomes, and differentiate between independent medical episodes and related sequences of events. By making these relationships explicit, the graph transforms raw temporal sequences into clinically meaningful narratives that support more effective decision-making.

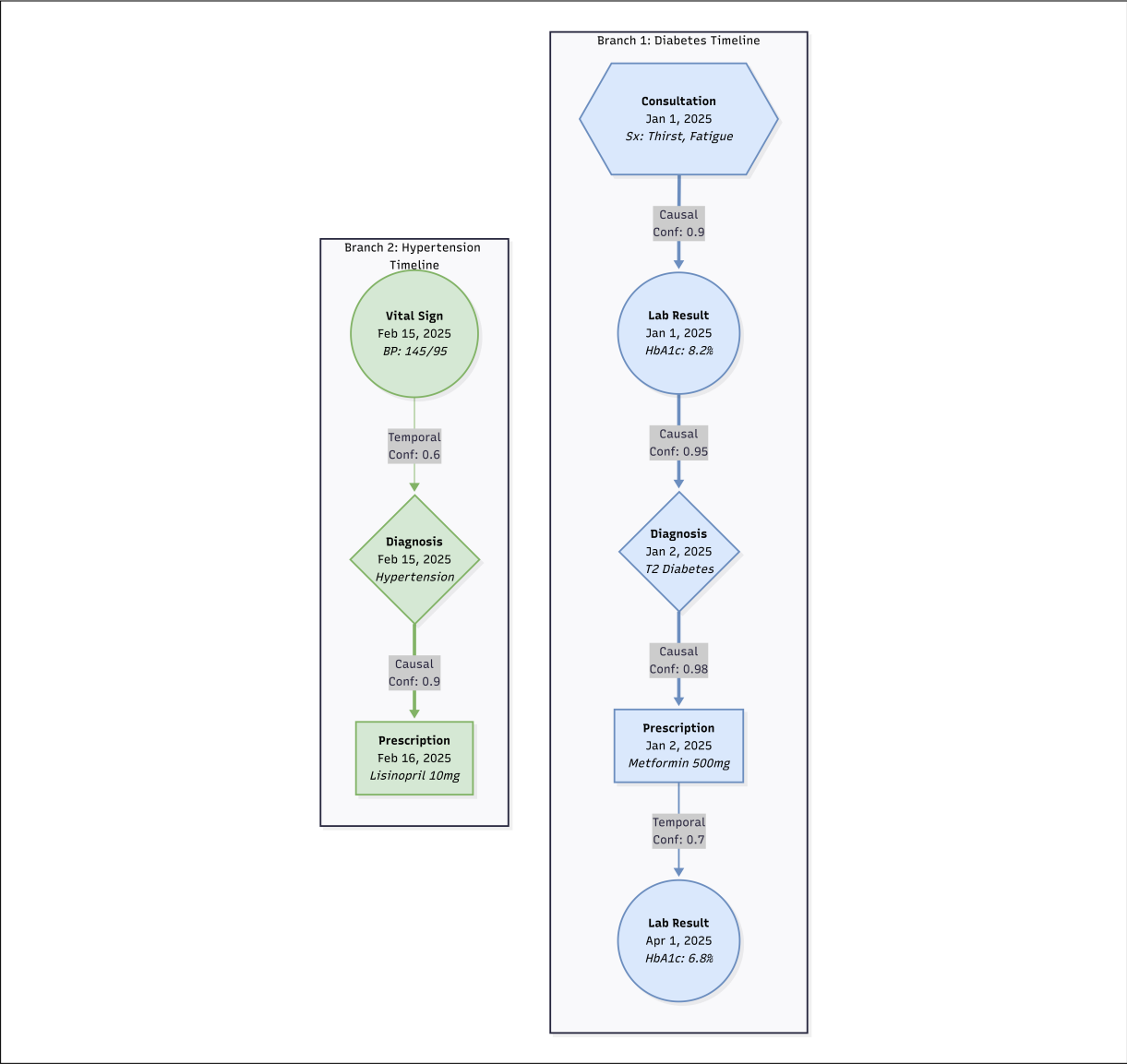


Figure 3.6: Example Patient History Graph showing a diabetes patient with concurrent hypertension, demonstrating temporal branching and multiple relationship types.

3.3.2 Graph Data Model

The Patient History Graph is built upon a relational foundation with two primary tables: `encounter_nodes` and `node_relations`.

Node Representation

Each node in the graph represents a distinct clinical event or observation. The system recognizes six primary node types: **Consultation** represents a clinical encounter between patient and clinician; **Diagnosis** denotes a confirmed medical condition recorded during or after a consultation; **Prescription** indicates a medication prescribed to address a specific condition; **LabResult** captures laboratory or diagnostic test results; **Vital** encompasses blood pressure, temperature, and other vital signs; and **Symptom** represents patient-reported subjective findings.

Each node stores rich metadata including its creation timestamp, an associated FHIR resource ID for traceability, its clinical category, and the patient to which it belongs. A fundamental design principle of the Patient History Graph is immutability: nodes, once created, cannot be modified or deleted. This architectural choice ensures that the graph preserves a complete and auditable record of the patient's medical journey, enabling clinicians to understand not just what happened, but also when records were created and by whom.

Edge Representation

Edges in the graph represent causal or temporal relationships. An edge from node *A* to node *B* indicates that event *A* preceded and potentially caused or informed event *B*. The system maintains an explicit `node_relations` table storing parent-child pairs, enabling efficient graph traversal for visualization and analysis.

3.3.3 Graph Construction Algorithm

The graph construction logic is encapsulated within the `historyGraphService`, a core backend service invoked whenever a new medical event is recorded. The algorithm operates in several stages:

Stage 1: Node Creation

When a new clinical event is received from the FHIR server (such as a new prescription, lab result, or consultation record), the system creates a corresponding node in the `encounter_nodes` database table. This node captures essential event information: the patient to which the event belongs, the clinical category of the event (Consultation,

Diagnosis, Prescription, etc.), the timestamp indicating when the event occurred, a reference ID linking back to the source FHIR resource for data provenance, and structured clinical details extracted from the FHIR resource including diagnosis codes, medication specifications, laboratory values, and other relevant clinical parameters.

Stage 2: Temporal Scanning and Parent Identification

After node creation, the algorithm performs a critical temporal backward scan of all existing nodes for the patient to identify potential parent nodes. This scan is bounded by three sets of clinical rules that determine what types of events can meaningfully be causally related. First, event type rules enforce domain-specific constraints: for instance, a Prescription node can only have a Consultation, Diagnosis, or another Prescription as a parent, while a Consultation cannot have a Prescription as a parent. These rules prevent semantically meaningless relationships from being created. Second, temporal window constraints ensure that relationships are only established between events within a clinically relevant time window. A prescription issued more than six months after a consultation is statistically unlikely to be causally related unless additional context suggests otherwise. Third, specificity matching ensures that the algorithm correctly associates events with their most clinically relevant predecessors. For example, a prescription for metformin (used in diabetes management) is linked to the most recent Diabetes diagnosis rather than to an unrelated hypertension diagnosis, even if both diagnoses exist in the patient's history.

The algorithm identifies the most recent "eligible" parent node based on these constraints. In cases where multiple candidates exist with similar temporal proximity, the system selects the most clinically specific match, prioritizing condition-specific diagnoses over generic symptoms or vice versa depending on the event context.

Stage 3: Edge Creation

Once a parent node is identified, a directed edge is created from parent to child in the `node_relations` table. The edge stores metadata including the relationship type (causal, temporal, or associative) and a confidence score indicating the algorithm's certainty in the relationship.

Algorithm Pseudocode

The following pseudocode formalizes the algorithm:

```
function constructGraphNode(newEvent, patientId):
    // Stage 1: Create node
    node = createNode(newEvent, patientId)
```

```

// Stage 2: Scan for parents
existingNodes = queryNodesByPatient(patientId)
candidateParents = []

for existingNode in existingNodes:
    if isValidRelationship(newEvent.type, existingNode.type):
        if isWithinTemporalWindow(newEvent, existingNode):
            if matchesClinicalContext(newEvent, existingNode):
                candidateParents.add(existingNode)

// Stage 3: Select and link parent
if candidateParents is not empty:
    parentNode = selectMostRelevantParent(candidateParents)
    createRelation(parentNode, node)
end if

// Invalidate cache
invalidateGraphCache(patientId)

return node
end function

```

3.3.4 Temporal Branching and Concurrent Episodes

Real-world patient histories are rarely linear. A patient may have multiple concurrent health issues being managed in parallel. For example, a patient may simultaneously receive treatment for diabetes, hypertension, and arthritis. In a traditional linear EHR, this complexity is flattened; all events appear chronologically without indication of which episode they belong to.

The CFI-Care graph model handles this through an explicit branching mechanism:

Head Node Concept

A "Head Node" is a node with no outgoing edges to other nodes in the current timestamp window. When a new clinical event arrives that does not fit within any existing causal chain (i.e., no valid parent is found), the system creates a new Head Node, effectively initiating a new branch in the history graph.

For example, consider a patient's history:

1. Consultation for chest pain (2025-01-01)

2. EKG test (2025-01-01)
3. Cardiology referral (2025-01-02)
4. New consultation for joint pain (2025-02-15) — **New Branch**
5. X-ray of knee (2025-02-15)
6. Rheumatology referral (2025-02-16)

The system recognizes that the joint pain consultation on 2025-02-15 cannot be causally linked to the earlier cardiology events (temporal gap exceeds threshold, clinical context is unrelated). Therefore, it creates a new Head Node for the rheumatology branch, allowing clinicians to visually separate the two episodes.

Visual Implications

From a visualization perspective, branching creates a multi-root forest structure (multiple separate trees within the graph). The frontend rendering engine positions these branches side-by-side or in hierarchical layout, enabling clinicians to track multiple concurrent conditions without visual clutter or confusion.

Clinical Advantages

Explicit branching provides significant clinical benefits that improve the utility of the Patient History Graph. Clinicians immediately recognize which events belong to which condition through visual separation, substantially reducing cognitive load compared to linearly ordered records. The system can provide condition-specific recommendations by analyzing the isolated branch rather than considering the entire patient history, enabling more targeted and relevant clinical decision support. Additionally, each branch can be summarized independently for specialist-to-specialist referrals, allowing specialists to focus on the specific problem domain without irrelevant context from other concurrent conditions.

3.3.5 Relationship Classification

Not all edges in the Patient History Graph carry the same semantic weight or clinical significance. The system classifies relationships into three distinct types to preserve this important distinction. Causal relationships represent situations where Node *A* directly caused or necessitated Node *B*. For example, a Diabetes Diagnosis directly necessitates a Metformin Prescription. These edges carry high confidence scores and strong clinical significance, indicating a clear cause-and-effect relationship. Temporal relationships occur when Node *A* preceded Node *B* within the same clinical episode, but causality is not

explicitly established. For instance, a Consultation followed by a Blood Pressure Reading taken during the same visit represents a temporal relationship rather than a causal one. Associative relationships capture scenarios where Node *A* and Node *B* are related through shared clinical context—such as both addressing the same underlying condition—but neither directly causes the other. For example, a High Cholesterol Lab Result and a Diabetes Diagnosis both present in the same patient might be associated through their shared metabolic dysfunction.

Each relationship in the graph stores a confidence score ranging from 0.0 to 1.0, generated by the construction algorithm based on how well the relationship satisfies the various constraint criteria. High-confidence relationships with scores exceeding 0.8 are displayed prominently in the visualization, while lower-confidence relationships are shown with reduced visual emphasis or as optional links that the clinician can explore if desired.

3.3.6 Handling Edge Cases

The Patient History Graph implementation must gracefully handle several challenging scenarios that arise in real-world clinical practice. Retroactive diagnosis occurs when a patient receives a diagnosis that applies to earlier symptoms after those symptoms have already been recorded. For example, a patient might experience chest pain on January 1st and only receive a diagnosis of coronary artery disease on March 1st. The algorithm detects this temporal mismatch and retroactively relinks the symptom node to the new diagnosis, maintaining causal clarity and preventing the false impression that the symptom was unexplained.

Conflicting information presents another challenge. When new FHIR data contradicts previously recorded information—such as a diagnosis being changed, retracted, or corrected—the system must handle this gracefully. Rather than deleting nodes (which would violate the immutability principle and destroy the audit trail), the system creates a new node representing the correction and links it appropriately to existing nodes. This approach preserves the complete historical record while accurately reflecting the current clinical understanding.

Circular references represent a potential structural problem. Although directed acyclic graphs (DAGs) cannot, by definition, contain cycles, the algorithm must validate that no accidental cycles are introduced during edge creation. This validation is enforced through topological sorting checks performed before persisting relationships to the database, preventing data consistency issues that could arise from cycles.

3.3.7 Query Patterns and Optimization

The frontend visualization layer requires efficient access to the graph structure through several common query patterns, each optimized for specific clinical use cases. Full Graph Retrieval fetches all nodes and edges for a patient, which is necessary during the initial

page load when clinicians first view a patient’s complete medical history. This operation is optimized through caching strategies discussed in Section 3.4. Ancestor Traversal answers questions such as ”What led to this prescription?” by fetching all ancestor nodes of a specific event. This pattern is implemented via recursive queries that are bounded by depth to prevent unbounded traversals. Branch Isolation allows clinicians to focus on specific medical episodes, such as ”Show only cardiology-related events,” by fetching all nodes within a specific branch. This operation uses topological partitioning to efficiently isolate branches without scanning the entire graph. Temporal Range Queries retrieve nodes within a specific date range, enabling clinicians to analyze disease progression over a specific time period. These queries leverage indexed timestamp columns to achieve $O(\log n)$ lookup time complexity, ensuring responsiveness even for patients with extensive histories.

3.4 Performance Optimization

To ensure low-latency responses for the frontend visualizations, the backend implements a specialized Two-Tier Caching Strategy using Redis. This hierarchical caching approach balances data freshness with response times, recognizing that different types of medical data have different update frequencies and computational costs.

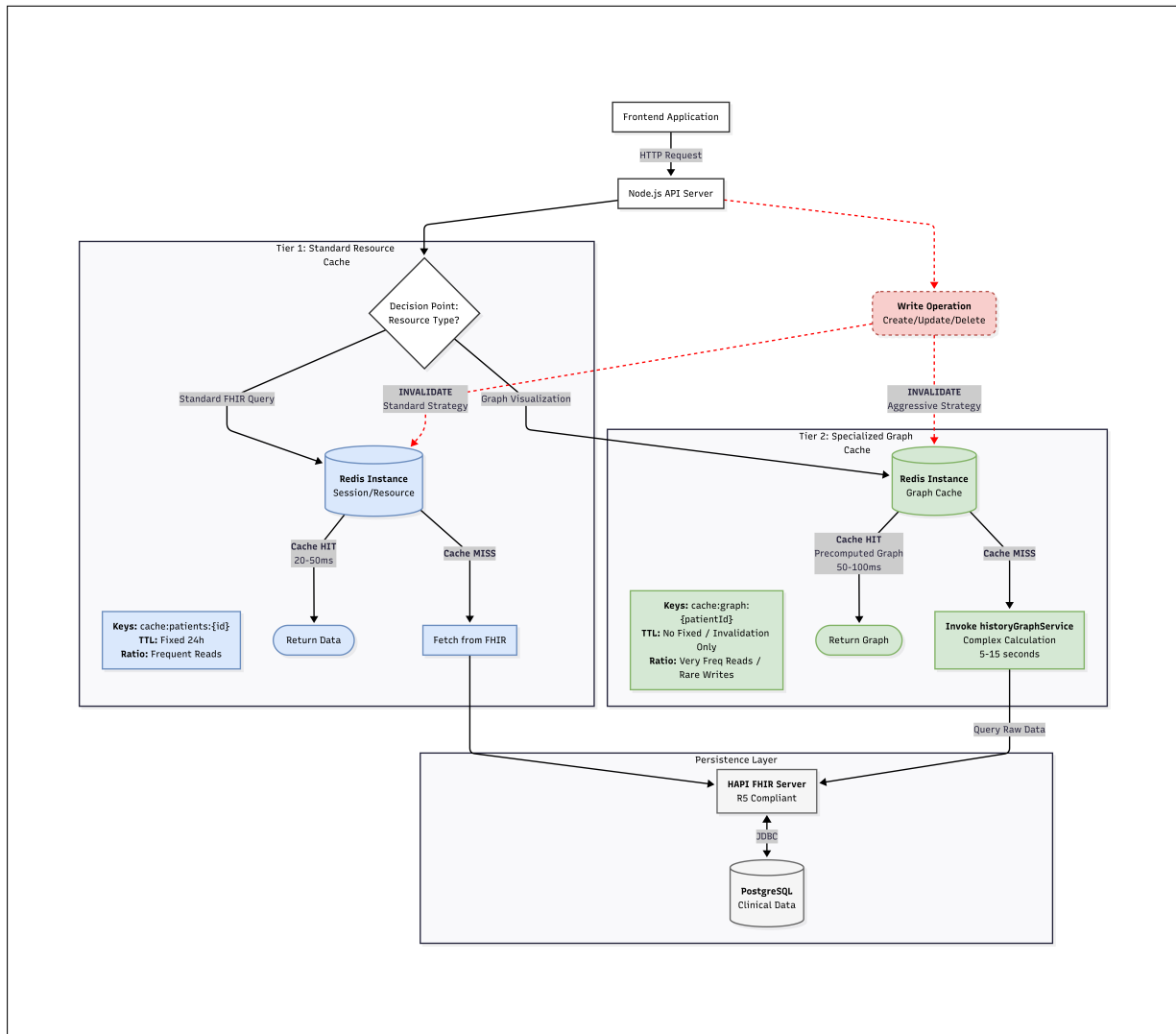


Figure 3.7: Two-Tier Caching Architecture showing distinct strategies for standard FHIR resources (Tier 1) and complex graph structures (Tier 2).

3.4.1 Tier 1: Standard Resource Cache

The first tier utilizes a generic `cacheHelper` middleware that intercepts HTTP requests for standard FHIR resources. This middleware operates at the request level, functioning as a transparent cache layer between the frontend client and the backend processing pipeline.

Mechanism and Interception

When an HTTP request arrives at the backend (such as a GET request to `/api/patients`), the `cacheHelper` middleware performs a sequence of operations to determine whether a

cached response can be served. The middleware first generates a deterministic cache key from the request URL and query parameters—for example, a request to `/api/patients?page=1&limit=20` generates the key `cache:patients:page:1:limit:20`. The middleware then queries the Redis cache for an entry matching this key. If a valid cache entry exists, meaning the entry has not expired, the cached response is immediately returned to the client, entirely bypassing the FHIR server, database query execution, and all downstream processing. This direct cache-to-client path reduces latency from potentially 500-2000ms (database query time) to sub-50ms (Redis in-memory retrieval). If no entry exists or the entry has expired, the request proceeds through the normal processing pipeline: the backend queries the FHIR server, processes the response, and returns it to the client. Before returning, the response is stored in Redis for future requests, repopulating the cache.

Time-To-Live (TTL) Management

Cached FHIR resources are assigned a default Time-To-Live (TTL) of 1 hour, representing a clinically acceptable window where cached data is considered sufficiently fresh for clinical use. This 1-hour window is chosen based on several practical considerations. From a clinical workflow perspective, most healthcare workflows operate on new fresh data every hour. A patient's medication list, diagnosis history, and vital signs are statistically unlikely to change multiple times within a single hour in most clinical scenarios. From a system load perspective, a 1-hour TTL significantly reduces the query volume to the underlying FHIR server and database, minimizing computational load during peak usage hours when many clinicians are accessing patient data simultaneously. While the 1-hour window does introduce some degree of data staleness, this tradeoff is clinically acceptable for read-heavy operations on relatively stable data that changes infrequently.

However, the TTL is configurable on a per-resource-type basis to accommodate different data freshness requirements. Real-time vital signs or active medication adjustments might use a shorter TTL of one hour to ensure clinicians see recent changes, while historical diagnoses that rarely change might use a longer TTL of 12 hours to reduce computational overhead.

Cache Invalidation for Tier 1

Although Tier 1 uses a fixed TTL-based expiration, certain write operations trigger explicit cache invalidation to ensure data consistency. When a patient record is modified—such as contact information changes or new allergies being recorded—the corresponding cache entries for that patient are immediately removed, forcing a fresh fetch on the next request. Similarly, when a new encounter is recorded or an existing one is updated, the cache entries for `/api/encounters` are invalidated to reflect the changes. The system maintains a comprehensive mapping of dependencies between cache keys and FHIR resources. When a resource is modified, all dependent cache entries are invalidated simultaneously, preventing scenarios where multiple cache entries contain stale references to the same resource.

Practical Impact

The practical benefit of Tier 1 caching becomes apparent in typical clinic workflows where healthcare providers request the same patient’s record multiple times during a single day. Without caching, each request would trigger a database query taking 500-1000ms, and checking the same patient 5 times would incur 2500-5000ms of cumulative latency. With Tier 1 caching in place, the first request incurs the full latency of 500-1000ms, but subsequent requests within 24 hours are served from cache in approximately 20-50ms each. For five checks of the same patient, the total latency becomes 500-1000ms plus 80-200ms for the cached requests, totaling 580-1200ms. This represents a 75-80 percent reduction in latency, dramatically improving the responsiveness of the system and reducing clinician wait times during patient data retrieval.

3.4.2 Tier 2: Specialized Graph Cache

The second tier is specifically designed for the computationally intensive Patient History Graph. While Tier 1 caches simple resource retrieval, Tier 2 caches the results of complex graph construction and relationship mapping.

Computational Cost of Graph Construction

Constructing a Patient History Graph from raw FHIR resources is computationally expensive for several interconnected reasons. Data aggregation requires retrieving all FHIR resources for a patient, including consultations, diagnoses, prescriptions, lab results, vitals, and symptoms, potentially from multiple FHIR repositories. A patient with an extensive medical history might have hundreds of such resources, necessitating multiple server calls and database queries. Relationship inference constitutes the most computationally intensive phase. For each new node, the algorithm must scan all existing nodes for that patient, evaluate whether causal relationships should exist by checking temporal windows, clinical context compatibility, and event type rules, compute confidence scores for each potential relationship, and determine the most relevant parent. This represents an $O(n)$ operation where n is the number of existing nodes, and complex patients with 500+ nodes can require significant computational resources. Graph formatting transforms the raw graph structure into a visualization-ready format suitable for the frontend rendering library. This transformation includes computing optimal node positions in the visual layout, determining visual attributes such as color, size, and emphasis, serializing the entire structure to JSON, and applying any additional formatting for the specific frontend framework. For a patient with 500 nodes, these cumulative operations can collectively require 5-15 seconds of computational time—far too slow for the responsive interactive experience that clinicians expect.

The Graph Cache Manager: `toonNodesCacheHelper`

The `toonNodesCacheHelper` service manages caching specifically for the Patient History Graph. Unlike the generic Tier 1 cache which uses simple TTL expiration, the graph cache is patient-specific and extensively optimized for the unique characteristics of graph data. Each patient has a dedicated cache entry with the key `cache:graph:{patientId}`, containing their complete precomputed graph structure. The cache value stores a comprehensive JSON object containing all nodes with their associated metadata (type, timestamp, clinical details), all edges with their relationship types and confidence scores, and precomputed visual attributes including node positions and branch assignments. Unlike Tier 1 caching which relies on fixed TTL expiration, the graph cache does not use time-based expiration at all. Instead, it relies entirely on explicit invalidation to ensure that cached data always reflects the current patient history.

Aggressive Invalidation Strategy

The graph cache employs an aggressive invalidation approach that prioritizes data consistency over computational efficiency. Any write operation to a patient's medical history immediately invalidates their corresponding graph cache entry. This design reflects a fundamental tradeoff in the caching strategy: write operations are relatively rare compared to read operations in clinical workflows, while read operations dominate. A single patient might have their graph read hundreds of times per day across multiple clinicians viewing the same patient from different workstations or different clinical contexts, yet the patient's medical history is modified only a few times per day. Given this asymmetry, it is computationally more efficient to invalidate the cache upon writes (an immediate operation) than to maintain cache staleness during reads or to implement complex update-in-place strategies. The invalidation is triggered by several categories of events: recording a new clinical event such as a consultation, diagnosis, prescription, or lab result; updating or correcting an existing node; modifying relationships between nodes; or retracting or changing a diagnosis or medication. Any of these operations immediately removes the cached graph, forcing reconstruction on the next request.

Cache Reconstruction Workflow

When a graph cache miss occurs, either due to invalidation from a write operation or because the cache has never been populated for that patient, a well-defined reconstruction workflow is executed. The frontend or backend first requests the graph for the patient. The `toonNodesCacheHelper` detects the cache miss and invokes the `historyGraphService` to reconstruct the graph from scratch. The reconstruction process retrieves all FHIR resources for the patient, executes the complete graph construction algorithm including temporal scanning and parent identification, and computes visual attributes. Once complete, the entire graph structure is serialized to JSON and stored in Redis. The fully reconstructed cache is then returned to the requesting client. Subsequent requests within

the same session or from other clinicians viewing the same patient retrieve the cached graph in sub-100ms latency, providing responsive interactive performance.

Hit Rate and Performance Impact

The effectiveness of the graph cache in improving system performance depends heavily on the read-to-write ratio, which varies depending on the clinical workflow pattern. In read-heavy workflows, which represent the typical clinical use case, a clinician views a patient's graph multiple times during a consultation, compares it with previous snapshots to track disease progression, and discusses findings with colleagues or consulting specialists. The same graph may be read 10-20 times before any modification to the patient record occurs. In these scenarios, the graph cache achieves hit rates of 85-95 percent, resulting in dramatic performance improvements. A graph that would normally require 15-20 seconds of computational time to construct is served from cache in 50-100ms, a 100-200x latency reduction. In contrast, write-heavy workflows are rare but do occur, such as when a clinician is entering comprehensive historical data for a newly-registered patient, recording 20-30 events that generate new medical events in quick succession. Each new event triggers cache invalidation, causing multiple cache misses and requiring full reconstructions. In these scenarios, the cache hit rate drops to 20-30 percent. The aggressive invalidation strategy in this case sacrifices caching efficiency to ensure that data consistency is always maintained. In typical clinical workflows, the combination of read-heavy access patterns and Tier 2 caching provides a 100-200x latency improvement for graph visualization, translating to substantially more responsive clinical interfaces and reduced clinician wait times.

Coordination Between Tier 1 and Tier 2

The two caching tiers work in close coordination to maximize performance across different types of requests. When the frontend requests standard FHIR resources such as a patient list or individual patient demographic data, Tier 1 cache is checked first, providing rapid responses for frequently accessed basic information. When the frontend requests the complex Patient History Graph visualization, Tier 2 cache is consulted, serving the pre-computed graph structure with all relationships and visual attributes already prepared. Invalidation operations propagate across both tiers simultaneously. For example, recording a new prescription invalidates both the Tier 1 cache entry for `/api/prescriptions` (which would now be stale relative to the 24-hour TTL) and the Tier 2 cache for that patient's history graph (which must be reconstructed to include the new node and relationships). This coordinated invalidation ensures that regardless of which API endpoint a clinician accesses, the data reflects the most recent state of the patient's record.

Memory Management and Cache Eviction

Redis is configured with a maximum memory limit to prevent unbounded growth and to integrate into production server resource constraints. To manage memory efficiently,

the CFI-Care backend employs an LRU (Least Recently Used) eviction policy. When Redis approaches its configured maximum memory capacity, the system automatically evicts the least recently accessed cache entries to make space for new data. This dynamic approach to cache management creates a natural working set where the cache automatically retains entries for patients with active clinical workflows (frequently accessed by multiple clinicians) while entries for inactive patients are evicted and subsequently reconstructed on-demand if and when those patients are accessed again. This ensures that memory usage remains bounded and predictable while simultaneously maximizing cache hit rates for the active subset of patients. In practice, this means that high-volume clinical workflows benefit from maximum cache coverage, while less frequently accessed patients gracefully degrade to on-demand reconstruction without requiring administrator intervention or manual cache management.

3.5 Frontend

article listings xcolor [english]babel caption graphicx [letterpaper,top=2cm,bottom=2cm,left=3cm,right=3cm,marginwidth=1cm]amsmath graphicx float [colorlinks=true, allcolors=blue]hyperref

3.6 Mobile

3.7 Problem Statement

The digital transformation of healthcare in Egypt faces significant structural challenges that directly impact patient care and safety. Despite national initiatives for modernization, the current healthcare ecosystem remains characterized by fragmentation, reliance on analog documentation, and a lack of standardized data governance. This thesis addresses these critical gaps by proposing a patient-centric mobile application designed to bridge the current digital fragmentation.

3.7.1 Data Fragmentation and the “Silo” Effect

A primary challenge facing Egyptian patients is the lack of interoperability between healthcare providers. Research indicates that Egyptian health entities operate in “silos,” utilizing discrete data collection systems that do not integrate or complement one another [?]. Consequently, patient data is scattered across various hospitals, private clinics, and diagnostic laboratories. In the absence of a unified national medical record, the patient is forced to assume the role of the sole custodian of their medical history. This places an undue burden on patients—particularly those with chronic conditions—to physically transport scattered paper reports and diagnostic films between providers, increasing the risk of information loss and discontinuity of care.

3.7.2 Reliance on Analog Systems and Data Quality

The persistence of paper-based (analog) data collection mechanisms significantly undermines data quality and accessibility. Current policy assessments reveal a fundamental lack of proper health-data collection mechanisms within the sector [?].

Collected low-quality data are characterized by incomplete fields, data entry errors, and reliance on manual updates.[?].

When transitioning between doctors, patients must rely on their memory to verbally relay complex medical histories. This often results in “information asymmetry,” where crucial clinical details—such as drug allergies or previous surgical interventions—are omitted, leading to potential misdiagnoses or redundant diagnostic testing.

3.7.3 Absence of Standardized Consent and Data Sovereignty

Furthermore, the current landscape lacks a robust framework for patient data rights and privacy. There is a notable absence of standardized consent practices; written consent for data usage is rarely obtained, and patients often lack awareness regarding how their health data is utilized or stored [?]. Patients currently possess no granular control over who accesses their records once they are handed over to a facility. This lack of transparency and control fosters a reluctance among patients to share sensitive health information, further hindering the effectiveness of clinical consultations.

3.7.4 The Need for Continuous Monitoring

Finally, traditional healthcare models in Egypt often fail to capture the patient’s daily health status. Current protocols rely on sporadic clinical visits, missing the continuous physiological data that modern wearable technology can provide. By failing to integrate real-time vitals into the formal medical record, the system misses critical opportunities for preventative monitoring and early intervention.

3.8 Mobile Application Goal and Methods

The primary objective of developing the proposed mobile application is to provide a secure, efficient, and patient-centered platform for managing personal health information within the digital medical ecosystem integrated with a FHIR-compliant backend.

3.8.1 Digital Document Management

The application enables patients to digitally capture, upload, and store medical reports, diagnostic scans, and other relevant health documents using their mobile devices. These documents are maintained within the application’s local storage and synchronized with the backend system when required, thereby reducing reliance on physical medical records.

3.8.2 Doctor-Patient Data Sharing

A core objective of the system is to support controlled and consent-based data sharing. Doctors may submit requests to access a patient's medical data, which the patient can explicitly approve or reject. Upon approval, patients are able to define granular access permissions, such as read-only or read-write access, as well as specify a time-bound duration for which the access remains valid. This approach ensures patient data privacy while enabling physicians to effectively review and update medical records when authorized.

3.8.3 Elimination of Physical Records

By digitizing medical documentation and access workflows, the system aims to eliminate common issues associated with traditional paper-based records, including loss, damage, and the burden of carrying extensive physical reports during medical visits.

3.8.4 Caregiver and Family Involvement

In addition to doctor access, the application supports caregiver and family member involvement. Patients may grant designated caregivers permission to monitor their health status, subject to explicit consent. Similar to doctor access, patients can define the level of access (read-only or read-write) and set temporal constraints for each caregiver, ensuring full control over shared medical information.

3.8.5 Appointment Management

The application further facilitates appointment management by allowing patients to schedule medical appointments directly through the mobile interface. During the booking process, patients may optionally attach relevant medical documents to provide physicians with preliminary information prior to consultation.

3.8.6 Wearable Device Integration

Finally, the system is designed to support integration with wearable and smart health devices, such as smartwatches, to collect physiological data including heart rate and other vital signs. This data can be incorporated into the patient's health profile to support continuous monitoring and more informed clinical decision-making.

3.9 Related Work

This section provides an overview of a selection of mobile applications comparable to the one in this work.

3.9.1 Apple Health

The closest existing commercial application to our proposed *AI-Powered Collaborative Patient-Centric Health Record Platform* is **Apple Health**. It serves as the global standard for patient-centric data aggregation. Below is an analysis of its core features which serve as a baseline for our project:

- **Centralized Health Hub:** It acts as a single repository for mobility data, sleep history, and vital signs collected from wearable devices. It also allows users to personalize their experience by storing demographics and basic health details [?].
- **Record Management:** Users can manually input data or download health records directly from supported healthcare institutions. Key features include searching for specific clinical categories (like allergies or vitals) and pinning important lab results for quick access [?].
- **Data Sharing & Privacy:**
 - *Provider Integration:* Users can share specific data points (e.g., heart rate trends) with doctors who view it on their own dashboards [?].
 - *Family Access:* The system allows for data sharing with trusted contacts (e.g., caregivers or family members) for monitoring, with granular control over what is shared [?].
- **Medication & Cycle Tracking:** Built-in modules allow for medication adherence logging and reproductive health tracking to predict fertile windows and track symptoms [?].

System Differentiation: The "Full Integrated Ecosystem"

While Apple Health excels as a *Patient-Side Aggregator*, our proposed system is fundamentally different because it is a **bidirectional, fully integrated ecosystem**.

The Missing Link: Doctor Web Interface

Apple Health relies on third-party integration; it does not provide the medical tools for the doctor itself. In contrast, our system includes a dedicated **Web Interface for Doctors**, creating a closed-loop environment:

- **Active Collaboration:** Unlike Apple Health, where data is passively "shared," our system allows doctors to actively write to the patient's record, with the patient consent, and update treatment plans.
- **Data shared with Doctor:** Our AI component does not just categorize data; it assists the doctor by analyzing the patient's inputs, to help the doctor in diagnosing the disease, a feature absent in standard health aggregators.

3.10 Market Analysis: The Egyptian Landscape

In the context of Egypt, our system addresses a significant gap. While there are successful health-tech players, none offer the complete *Full Integrated Ecosystem* we are proposing.

3.10.1 Vezeeta

- Vezeeta currently has around 1 million users. The application can be downloaded for free in the Google Play-Store and the Apple Store.
- *Focus:* Primary focus is on **booking and telemedicine**.
- *Limitation:* These platforms function primarily as marketplaces. They do not typically offer a persistent, longitudinal health record that stays with the patient across different providers, nor do they utilize advanced AI, used by doctors, for clinical decision support during the collaboration.

3.10.2 Lumi by Nextcare:

- *Market Reach:* A widely used application in the region with over 1 million downloads [?].
- *Focus:* Specialized primarily in insurance management, approvals, and pharmacy benefits, with some telehealth features.
- *Limitation:* It lacks a "full closed system" for clinical management. Crucially, it does not structure medical history as a connected longitudinal record. Documents and visits are often treated as isolated transactional events (e.g., a claim or a prescription) rather than a continuous medical narrative.

3.10.3 Our Competitive Advantage: The "Medical History Tree"

The most distinct differentiator of our system is the data architecture, designed to solve the fragmentation found in apps like Lumi:

- **Graph Visualization (Doctor's View):** Unlike existing solutions that show simple lists, our Web Interface presents the patient's history as a **Graph/Tree Structure**. Doctors can traverse this graph where each "node" represents a specific medical event (visit, procedure, or test) containing all attached documents and AI summaries. This allows for rapid correlation between past and present conditions.
- **Major Events Timeline (Patient's View):** On the mobile application, the patient sees a simplified, chronological view of "Major Events." Each event expands into a detailed node containing the specific visit info and documents, ensuring they own a complete, navigable copy of their medical history, rather than just a list of insurance claims.

3.11 Flutter and Dart

3.11.1 Clean Architecture in the Mobile Application

To ensure a scalable, maintainable, and testable codebase for the patient mobile application, this project adopts principles of Clean Architecture.

Clean Architecture is a layered software design paradigm that promotes clear separation of concerns, dependency management, and modularization of application logic. The fundamental goal of this approach is to isolate core business rules from implementation details and platform-specific frameworks, thereby improving long-term maintainability and facilitating unit testing [?].

According to the Flutter documentation on application architecture, well-structured applications organize code into logical layers that separate the user interface, application logic, and data handling. The documentation emphasizes the importance of isolating state management, dependency injection, and data access concerns to support growth and complexity in larger applications [?].

As illustrated in Figure 3.8, the application is structured into three principal layers: the presentation layer, the domain layer, and the data layer. Each layer has a distinct responsibility and communicates with adjacent layers through abstract interfaces, ensuring that dependencies always point inward toward business logic rather than outward toward infrastructure or frameworks [?].

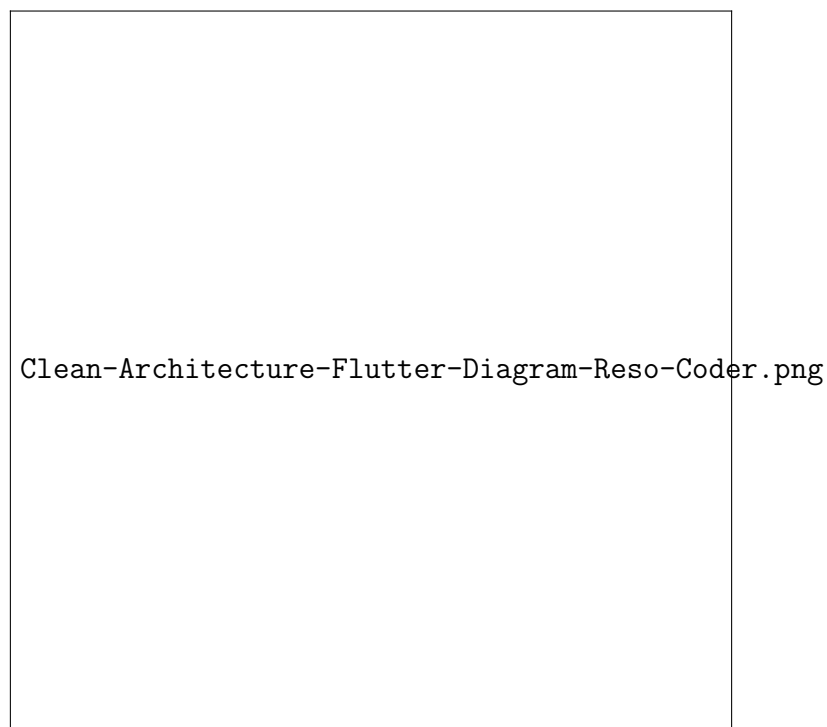


Figure 3.8: Layered Clean Architecture adopted in the mobile application

3.11.2 Layered Structure

Presentation Layer This outermost layer contains code that directly interacts with the user interface. In the context of this Flutter application, it includes widgets, screens, routing logic, and Riverpod providers to manage application state when implemented. The presentation layer orchestrates the flow of data from use cases in the domain layer to the UI and captures user interactions for processing. Presentation logic is kept distinct from business rules and data access to avoid tightly coupling UI changes with core system logic [?].

Domain Layer The domain layer encapsulates the core business logic and entities of the system. It is independent of Flutter or any external dependencies. This layer defines use cases that represent application-specific operations, such as requesting document access or scheduling appointments, and serves as the central coordinator of business rules. The domain layer's independence from external libraries and frameworks enhances testability and ensures that business logic can be validated in isolation [?].

Data Layer The data layer is responsible for integrating with external systems, including APIs, local storage, and the backend FHIR server. It provides implementations of repository interfaces defined in the domain layer and manages data transformation between network responses, local models, and domain entities. The layered separation ensures that external dependencies do not leak into the domain or presentation layers [?].

3.11.3 Centralized Runtime Permission Handling

To ensure a secure and seamless interaction with device hardware, the application implements a centralized asynchronous mechanism for managing runtime permissions. This approach abstracts the complexity of the `permission_handler` library and follows established patterns for robust permission management [?]. The function `handlePermission` provides a unified interface for requesting access to sensitive resources such as the camera, storage, or microphone.

The function operates as a `Future<bool>`, returning a boolean value to indicate the success or failure of the authorization process. Upon invocation, the function executes the following logic flow:

1. **Asynchronous Request:** It initiates a request to the underlying operating system using `permission.request()`. The execution yields until the user interacts with the system dialog or the operating system returns the current status.
2. **Status Evaluation:** The returned `PermissionStatus` is evaluated against standard authorization states:

- **Granted:** If the user approves the request (`isGranted`), the function immediately returns `true`, allowing the calling process to proceed.
- **Permanently Denied:** If the permission is flagged as `isPermanentlyDenied` (a state where the operating system suppresses further prompts due to repeated denials), the system intervenes to prevent a silent failure. It displays a toast notification instructing the user to enable the permission manually and invokes `openAppSettings()` to redirect the user to the device's settings menu.
- **Denied:** In cases of a standard denial, a feedback message is displayed via `Fluttertoast` to inform the user of the restriction, and the function returns `false`.

By encapsulating this logic, the system ensures that permission failures are handled gracefully, guiding users through recovery steps when necessary rather than causing application crashes or unresponsive features. The implementation of this logic is demonstrated in Listing ??.

```
[ language=Java, basicstyle=, frame=single, breaklines=true, caption=Asynchronous Permission
Handling Function, label=lst:permlogic]import'package : permission_hander/permission_hander.dart';import'pac
fluttertoast/fluttertoast.dart';

Future<bool> handlePermission( Permission permission, String name, ) async final status
= await permission.request();

// Case 1: Permission Granted if (status.isGranted) return true;

// Case 2: Permanently Denied (Requires Settings) if (status.isPermanentlyDenied) Fluttertoast.
msg: 'namepermissionpermanentlydenied.Enableitinsettings.',);openAppSettings();returnfalse;

// Case 3: Standard Denial Fluttertoast.showToast(msg: 'namepermissiondenied');returnfalse;
```

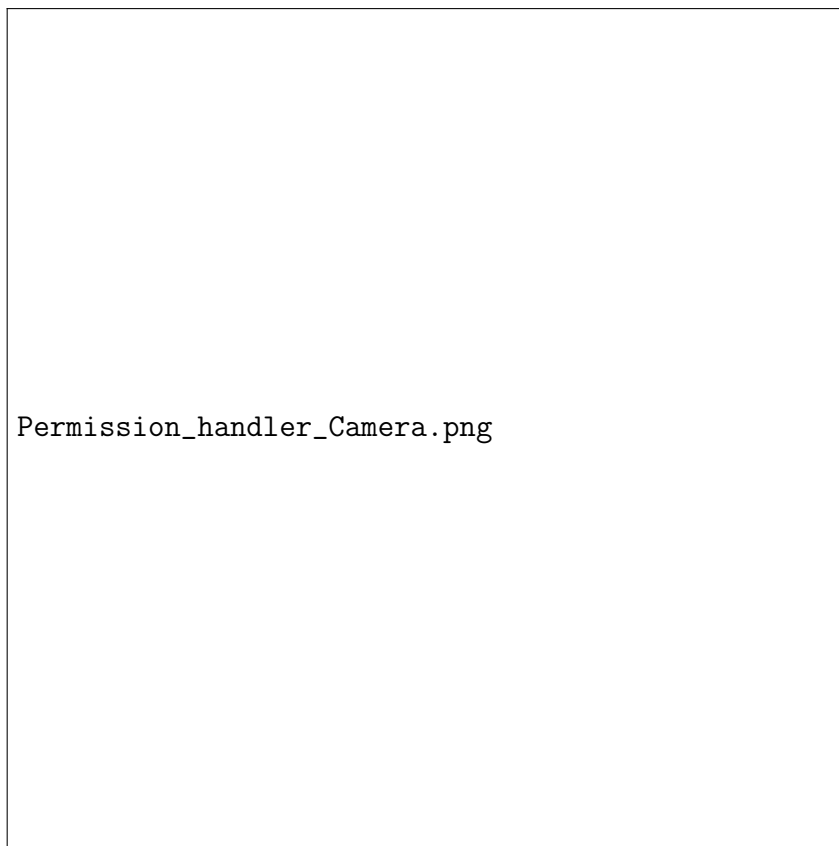


Figure 3.9: using handlePermission For Camera Access In code

Best Practices for handling Permissions in Flutter

- **Contextual Permission Requests:** To avoid overwhelming the user and to establish context, the application rejects the traditional “up-front” permission model (where all permissions are requested at launch). Instead, a *Contextual Request* pattern is employed. Permissions are only requested at the specific moment a feature is invoked. For example, access to the **Camera** is requested solely when the user attempts to scan a medical report.

- **Transparent Rationale and Trust Building:** In compliance with privacy best practices, the application explains the “Why” before the “How.” Prior to triggering the system permission dialog, the application presents a custom UI explanation detailing exactly why a specific permission is needed and how it benefits the user. This transparency is critical in a healthcare context, increasing the likelihood of approval and reinforcing the patient’s understanding that their data is being used strictly for medical management.[?]
- **User Agency and Graceful Degradation:** The system is designed to respect user choices without compromising the core functionality of the application. If a user denies a non-critical permission, the app enters a state of *Graceful Degradation*. The application avoids repetitive prompting (“nagging”), ensuring a respectful user experience. [?]
- **Comprehensive State Handling:** The application logic accounts for the full lifecycle of permission states, ensuring stability across different scenarios[?][?]:
 - **Granted:** The feature proceeds immediately.
 - **Denied:** The feature is disabled, and the user is informed of the limitation.
 - **Permanently Denied:** In cases where the user has selected “Don’t ask again,” the application detects this state and provides a subtle option to guide the user to the system **App Settings** menu for manual enablement, should they choose to do so later.

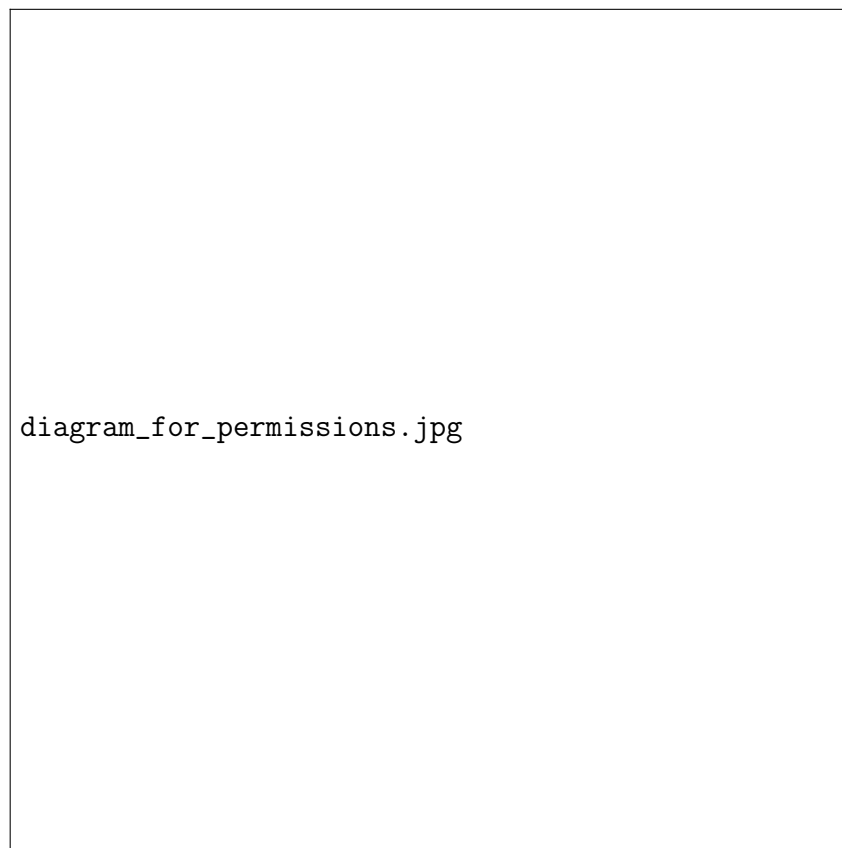


Figure 3.10: Best Practices for Handling Permissions in Flutter

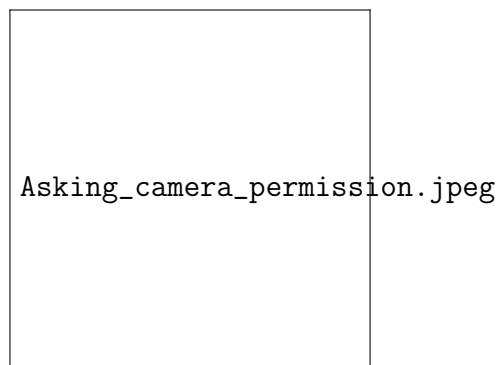


Figure 3.11: Dialog to allow the app to access the camera

3.12 Requirements Engineering

To ensure the delivery of a viable and effective application, the development lifecycle begins with a stakeholder analysis to establish the system context. This is followed by a comprehensive definition of the functional, non-functional, and optional requirements.

Synthesizing this information facilitates the creation of a domain model, which acts as the foundational blueprint for the subsequent database design.

3.12.1 Target Group and User Group

The mobile application is designed for a broad demographic, targeting two key user segments:

- **Primary Users (Patients):** Individuals seeking a digital solution to store and organize their medical history, accessible via any standard smartphone.
- **Secondary Users (Caregivers/Family):** Family members or guardians who require shared access to monitor the patient's health status and assist in managing their care plan.

By digitizing medical records, the application serves anyone looking to transition from fragmented paper records to a structured, accessible mobile archive.

3.12.2 Stakeholder

Stakeholders are categorized into Internal Stakeholders, who are directly involved in the development and operation of the system, and External Stakeholders, who utilize the system's end products.

External Stakeholders

External stakeholders are the primary users of the system. Their interaction drives the application's core functionality.

1. Patients (Primary Users) Interaction:

- **Data Management:** Patients interact with the mobile interface to upload medical documents (lab results, scans) and view their historical data presented in a structured "Major Events" timeline.
- **Access Control:** They possess full authority over their data privacy. Patients receive requests from doctors for data access and can grant permissions (Read-Only or Read-Write) for a specific duration. Crucially, they retain the ability to revoke this access at any time.
- **Family Management:** Patients can authorize family members to view their health records for monitoring purposes.

Benefits:

- **Centralized History:** Eliminates the need for physical paperwork by consolidating all medical history into a single, organized mobile repository.
- **Data Ownership:** Empowers patients with absolute control over who views their medical information and for how long.
- **Continuity of Care:** Ensures that any new doctor can immediately access a complete medical background, reducing diagnostic errors.

2. Family Members (Secondary Users) Interaction:

- **Monitoring:** Through shared access granted by the patient, family members can view health trends and documents to assist in care.
- **Administrative Support:** Unlike doctors, family members have the permission to book appointments on behalf of the patient, facilitating care for elderly or incapacitated users.

Benefits:

- **Remote Assurance:** Provides peace of mind by allowing real-time monitoring of a loved one's health status.
- **Care Coordination:** Simplifies the logistical burden of managing appointments and records for dependents.

3. Doctors (Service Providers) Interaction:

- **Session Initiation:** While doctors primarily use the Web Interface, their actions directly trigger updates on the mobile app. A doctor initiates a "New Encounter Node" in the patient's history tree during a visit.
- **Access Requests:** Doctors request permission to view a patient's historical records. If granted "Write" access, they can add new diagnoses and prescriptions to the encounter node.
- **Limitation:** Doctors strictly cannot book appointments; this function is reserved for patients and their families to prevent conflict of interest or scheduling errors.

Benefits:

- **Informed Decision Making:** Instant access to a verified, comprehensive medical history allows for more accurate diagnoses.
- **Efficiency:** Reduces time spent interviewing patients for past medical details, allowing more focus on current treatment.

Internal Stakeholders

Internal stakeholders are responsible for the technical construction, maintenance, and security of the platform.

1. Mobile Application Developers (Flutter Team) Interaction:

- **Frontend Implementation:** Responsible for building the cross-platform mobile interface using the Flutter framework.
- **API Integration:** They implement the communication layer that sends authentication requests and data queries to the backend.
- **UX/UI Design:** They translate the complex "Medical History Tree" concept into a user-friendly mobile timeline.

2. Backend & System Architects Interaction:

- **Authentication Flow:** They manage the identity management integration. The Flutter app sends credentials to the Node.js backend, which acts as an intermediary to communicate with **Keycloak** for secure Single Sign-On (SSO) and token validation.
- **Data Orchestration:** They maintain the Node.js middleware that fetches data from the **FHIR Server** and formats it for the mobile client.
- **Standardization:** They ensure that all medical data stored and retrieved adheres to HL7 FHIR standards for interoperability.

3.13 System Context Diagram



Figure 3.12: Sequence Diagram for Mobile Application

3.14 System Requirements Specification

The requirements for an application can be divided into three categories: functional requirements, non-functional requirements, and optional requirements. In this section all 3 requirements will be discussed.

3.14.1 Functional Requirements

The functional requirements define the specific behaviors and functions of the mobile application.

Patient Capabilities

- **FR-01 Document Management:** The system shall allow patients to upload, categorize, and tag medical documents (e.g., Lab Results, Radiography) into their personal repository.
- **FR-02 History Visualization:** The application shall present medical history as a "Major Events" timeline. Each event must be expandable to reveal detailed encounter notes and attached files.
- **FR-03 Access Control Management:**
 - The system shall notify patients of incoming access requests from doctors.
 - Patients shall be able to grant *Read-Only* or *Read-Write* permissions.
 - Patients shall be able to define the duration of access and revoke permissions immediately at any time.

Family & Caregiver Capabilities

- **FR-04 Monitoring Dashboard:** Authorized family members shall have a read-only view of the patient's medical timeline to monitor health trends.
- **FR-05 Proxy Booking:** Unlike doctors, family members shall have the authority to search for providers and schedule appointments on behalf of the patient.

3.14.2 Non-Functional Requirements

- **NFR-01 Interoperability:** The application must parse and render data compliant with HL7 FHIR standards to ensure seamless integration with the Node.js backend.
- **NFR-02 Localization:** To serve the local demographic, the interface must fully support Arabic (RTL) and English (LTR) languages.
- **NFR-03 Performance:** The medical history timeline uses "Lazy Loading" to ensure the application remains responsive even for patients with extensive medical records.

3.14.3 Optional Requirements (Future Scope)

- **OR-01 Medication Adherence System:** Automated push notifications to remind patients of medication schedules derived from their prescriptions.
- **OR-02 Wearable Device Sync:** Integration with native health APIs (Google Fit / Apple Health) to import real-time vitals like heart rate and step count.

3.15 Use Cases

This section describes the primary interactions between the users (Actors) and the mobile application system.

3.15.1 SignIn and SignUp Pages

Actors: Patient, Family Member

Description: The entry point for identifying users and securing the session. Given the sensitive nature of medical data, this process integrates with Keycloak for secure identity management.

- **Sign Up:** New users register by providing personal details (Name, National ID, Email) and creating a secure password. The system creates a new FHIR Patient resource on the backend upon successful registration.
- **Sign In:** Registered users authenticate using their credentials. Upon success, the app retrieves a secure access token to authorize subsequent API requests.

3.15.2 Document Management

Figure 3.13 illustrates the patient's interactions with their personal medical history data.

1. **View Document List:** The user requests to see their existing medical files. This action requires the system to query the database using the user's secure authentication token.
2. **Upload Document:** The user adds new records to their profile. This involves selecting a file and assigning title.
3. **Scan Document:** The user adds scans new records to their profile. This involves giving camera permission and scan the file and assigning title. Scanned Documents can be accessed through file manager in the phone.

4. **Refresh List:** Upon a successful upload, the system automatically triggers a refresh of the document view to reflect the new entry immediately.



Figure 3.13: Documents Management Use Case

3.15.3 Medical History Navigation

Figure 3.17 illustrates how a patient can navigation to explore a their medical history.

1. **View Major Events (Dashboard):** The entry point where the patient views high-level summaries of their active medical contexts (e.g., Cardiac Workup).

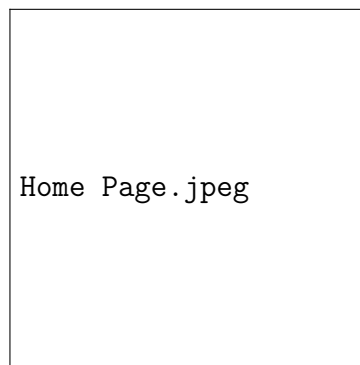


Figure 3.14: Major Events

2. **View Event Nodes:** By selecting a specific major event, the user extends their interaction to view a chronological list of related encounters (the "Nodes").

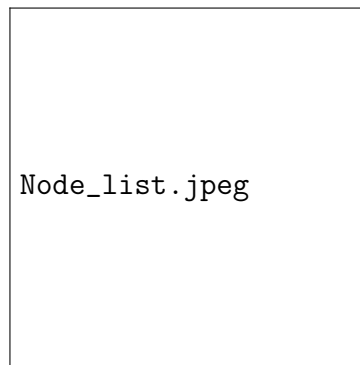


Figure 3.15: List of Nodes

3. **View Encounter Details:** Selecting a specific node allows the user to inspect the full clinical details of that visit.
4. **View Attached Documents:** Within the details view, the user can optionally retrieve and view specific medical files (PDFs/Images) associated with that encounter.

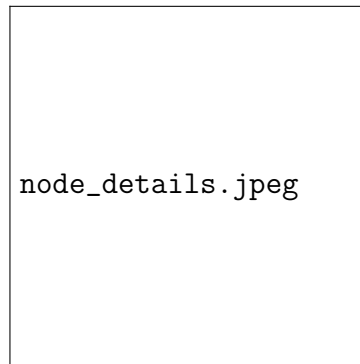


Figure 3.16: Node Details



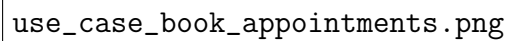
Figure 3.17: History Navigation Use Case

3.15.4 Appointment Booking

Figure 3.18 depicts the functional capabilities available to the Patient during the scheduling process.

1. **Core Functionality:** The primary goal is the *Book Appointment* use case. To complete this, the system mandates several included steps: searching for a specialist, selecting a valid time slot, and confirming the details.

2. **Optional Extension:** The *Attach Documents* use case is modeled as an extension. It is not required to complete a booking but provides optional value (sending scans/PDFs) upon successful completion.
3. **Actors:** The *Patient* is the primary initiator. The *Family Member* also inherits these capabilities, allowing them to perform the same actions on behalf of the patient.



use_case_book_appointments.png

Figure 3.18: Booking Appointments Usecase

3.16 My Profile Update

Figure 3.19 details the data consistency mechanism.

1. **Local Commit:** When the user saves changes, the application first updates the local SQLite database. This ensures the user interface is responsive and works even with poor internet connectivity.
2. **Remote Synchronization:** Immediately following the local commit, the application initiates an asynchronous API call to update the backend server.
3. **Consistency Check:** Upon a successful server response, the application confirms the synchronization status, ensuring the patient's data is consistent across all platforms.

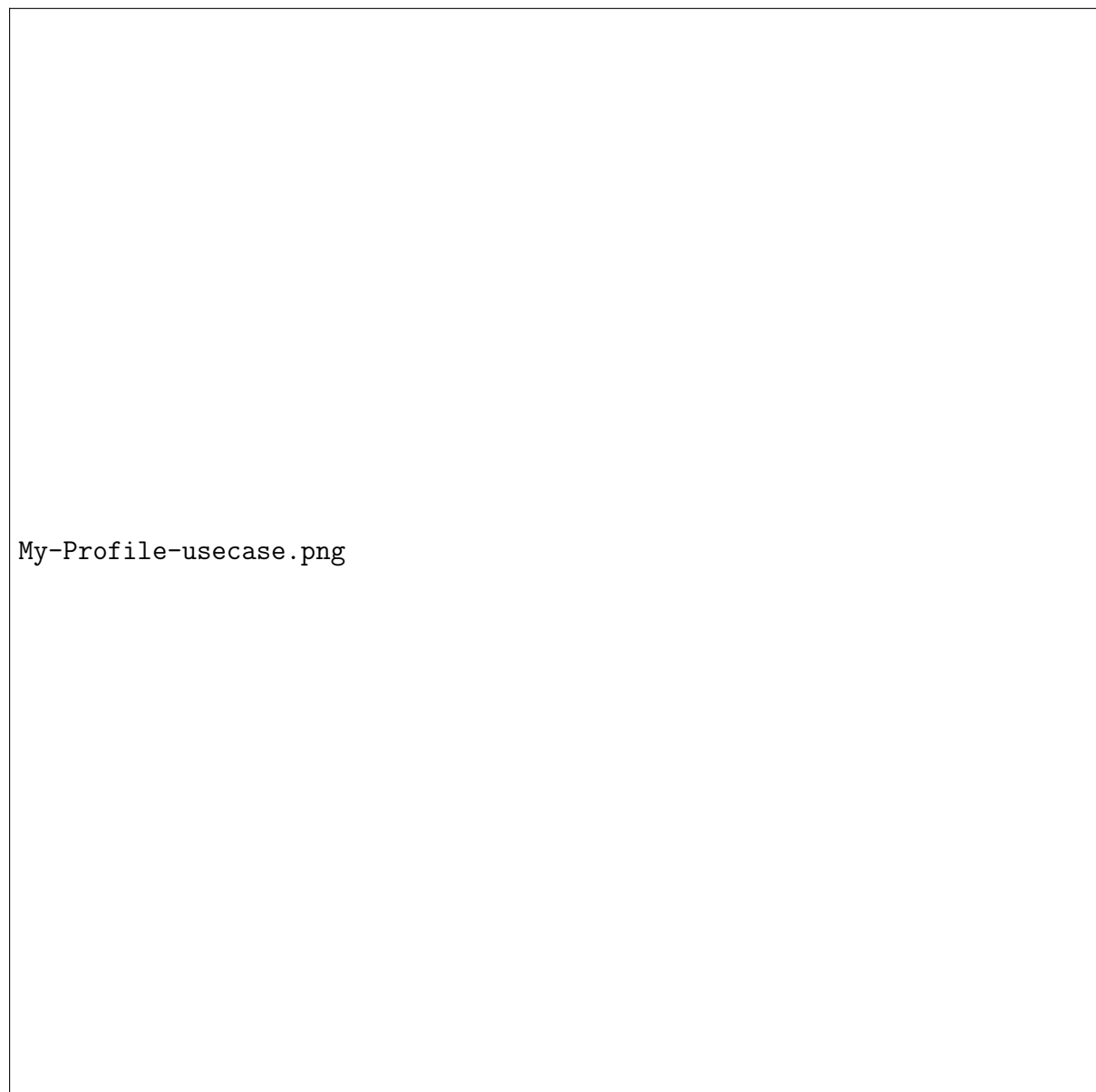


Figure 3.19: My Profile Use Case

3.17 Quality Assurance and Testing

3.17.1 Unit Testing

Unit testing is the process of verifying the correctness of a single function, method, or class in isolation. It is considered the foundation of the testing pyramid. The primary goal is to ensure that the business logic and algorithms function as expected under a variety of conditions without any interaction with the User Interface (UI) or external dependencies such as databases or web servers [?].

As described by Torres [?], unit tests help ensure that specific pieces of logic produce the expected output given a particular input. These tests are characterized by their execution speed and low maintenance cost. In Flutter, unit tests typically utilize the `test` package to define expectations and assertions for Dart classes.

3.17.2 Widget Testing

Widget testing, often referred to as component testing in other frameworks, sits between unit and integration testing. A widget test verifies the behavior of a single widget or a small group of widgets [?].

According to Bharti [?], the objective of widget testing is to ensure UI reliability and quality by asserting that the widget’s visual layout is rendered correctly and that it responds appropriately to user interactions, such as button taps or text entry.

Unlike unit tests, widget tests require a simulated environment. The Flutter framework provides the `WidgetTester` utility, which allows developers to build and interact with widgets in a test environment that is significantly faster than a full UI system but complex enough to render the widget tree [?].

3.17.3 Integration Testing

Integration testing involves testing the application as a complete system. This methodology validates that all individual components—including the UI, business logic, and external services—work together correctly [?].

These tests simulate a real user’s journey through the application, such as logging in, navigating through screens, and performing data updates. Because integration tests typically run on real devices or emulators to capture accurate performance data and visual rendering, they are generally slower and more resource-intensive than unit or widget tests.

3.17.4 Testing Architecture and Tools

We utilized the following standard Flutter testing libraries:

- **flutter_test**: The core framework used to write and run assertions.[?]
- **mockito**: Used to simulate external dependencies. Since our ViewModels depend on Repositories (e.g., `BookingRepository`) and Plugins (e.g., `ImagePicker`), we created "Mock" objects to return controlled dummy data during tests.[?]
- **build_runner**: Used to automatically generate the Mock classes (e.g., `MockMajorEventRepository`) using the `@GenerateMocks` annotation, ensuring our test mocks stay up-to-date with our actual interfaces.[?]

- **WidgetTester:** Provided by the `flutter_test` package, this allows us to build and interact with widgets in a test environment without running the full app on an emulator.[?]
- **network_image_mock:** Since widget tests run in a restrictive environment without real internet access, standard `Image.network` widgets would fail. We used this package to wrap our tests and mock successful image loads.[?, ?]
- **Provider Injection:** Many of our widgets (e.g., `DoctorCard`) rely on `ChangeNotifierProvider`. In tests, we injected "Mock Providers" (created with Mockito) to supply controlled data and verify that buttons trigger the correct business logic.[?]

3.17.5 Unit Testing Implementation

To ensure the reliability of the application's business logic and maintain a stable code-base, we implemented a comprehensive Unit Testing strategy. This strategy focused on verifying the behavior of our **ViewModels** (Providers) in isolation from the UI and external dependencies (such as the backend API or local device storage).

Booking Logic (`BookingProvider`)

The booking process involves complex state management where selecting a doctor resets previously selected dates to prevent invalid slots.

- **Scenario Tested:** We verified that selecting a new doctor automatically clears the `selectedDate` and `selectedTime` to enforce valid scheduling.
- **Optimistic Updates:** We tested the `confirmBooking` method to ensure that once the repository returns success, the local appointment list updates immediately without requiring a full page refresh.

```
// Code Snippet: Verifying Booking Reset Logic (booking_provider_test.dart)
test('Selecting a doctor should reset Date and Time', () {
  // Arrange: Set a date first
  provider.setTimeSlot(DateTime(2026, 1, 1), '10:00 AM');

  // Act: Select a new doctor
  provider.selectDoctor(createDummyDoctor());

  // Assert: Doctor is set, but time/date are wiped (null)
  expect(provider.selectedDoctor, isNotNull);
  expect(provider.selectedDate, null);
  expect(provider.selectedTime, null);
});
```

Document Uploads (DocumentAddViewModel)

This module interacts with native platform channels (File Picker, Image Picker). Testing it requires intercepting these platform calls.

- **Validation Logic:** We verified that the `isValid` getter returns `false` unless both a "Title" and a "File" are present, preventing users from submitting incomplete forms.
- **Platform Mocking:** Since Unit Tests run on a computer (not a phone), calls to `getApplicationDocumentsDirectory` would normally crash. We used `MethodChannel` mocking to intercept these calls and return a temporary directory on the test machine.

Medical History Timeline (MajorEventProvider)

This provider handles fetching the patient's "Major Events" and their sub-nodes.

- **State Management:** We tested the `fetchEvents` method to ensure the `isLoading` boolean correctly toggles to `true` when the call starts and `false` when data is received.
- **Data Clearing:** We verified that fetching details for a new event (e.g., "Surgery") correctly clears the old data from the previous event to prevent showing stale information.

```
// Code Snippet: Mocking Repository Responses (major_event_provider_test.dart)
test('fetchEvents success updates list and loading state', () async {
  // Arrange: Stub the repo to return a dummy list immediately
  when(mockRepo.getMajorEvents())
    .thenAnswer((_) async => [createDummyEvent()]);

  // Act: Trigger the fetch
  final future = provider.fetchEvents();

  // Assert: Loading should be true while fetching
  expect(provider.isLoadingEvents, true);

  await future;

  // Assert: Data is populated and loading is false
  expect(provider.events.length, 1);
  expect(provider.isLoadingEvents, false);
});
```

Test Coverage Results

By isolating the ViewModels and mocking the repositories, we achieved high test coverage for the application’s core logic. This approach allowed us to:

1. **Identify Edge Cases:** We caught issues where invalid form submissions were possible before implementing the validation checks.
2. **Refactor Safely:** We could optimize the `BookingProvider` logic with confidence, knowing the tests would fail if we broke the ”reset date” functionality.
3. **Improve Efficiency:** Running these unit tests takes less than 2 seconds, compared to manual testing which would require navigating through the entire app.

3.18 Future Work

3.18.1 Push Notifications

To enhance user engagement and ensure immediate response to critical events (e.g., doctor access requests), we plan to implement a robust Push Notification system. This architecture relies on **Firestore Cloud Messaging (FCM)** as the delivery intermediary.

Technical Implementation The proposed workflow consists of two distinct phases: Token Registration and Message Delivery.

- **Token Generation (Frontend):** Upon logging in, the Flutter application utilizes the `firebase_messaging` [?] package to generate a unique Registration Token for the device. This token acts as the specific address for the user’s phone.
- **Token Registration (Backend):** The mobile application transmits this token via a secure API endpoint (e.g., `POST /user/fcm-token`) to our Node.js backend. The backend associates this token with the User ID and stores it in the database.
- **Notification Triggering:** When a business event occurs (e.g., a doctor requests access), the Node.js backend retrieves the target user’s stored FCM token. It then utilizes the `firebase-admin` SDK to dispatch the payload to the FCM servers, which subsequently route the notification to the user’s device, even if the application is in the background.[?]

3.18.2 Wearable Data Integration

To transition from a system of historical records to a platform for active health monitoring, we propose integrating **Android Health Connect**. This allows the application to ingest vital signs collected by smartwatches and fitness trackers.

3.18.3 Objective: Remote Patient Monitoring (RPM)

The goal is to automatically populate the "My Vitals" dashboard without manual entry. By syncing with devices (e.g., Samsung Galaxy Watch, Google Pixel Watch), the system can track:

- **Heart Rate & Variability (HRV):** To detect potential cardiac anomalies early.
- **Activity Levels (Steps):** To monitor recovery progress after surgery.
- **Sleep Quality:** To assess overall patient well-being.

3.18.4 Health Connect

The Health Connect app is responsible for handling all the requests that your application sends through the Health Connect SDK. These requests include storing data and managing its read and write access. [?]

1. **Data Aggregation:** Smartwatches sync their raw sensor data to the Health Connect app on the patient's Android phone.
2. **Privacy-First Access:** Our Flutter application requests specific granular permissions (e.g., `READ_HEART_RATE`) from Health Connect.
3. **Background Synchronization:** A background worker in the Flutter app periodically fetches new data points and transmits them to our Node.js backend.
4. **Anomaly Detection:** The backend analyzes these trends and triggers an alert notification to the doctor if vitals exceed safety thresholds.

Bibliography